

EMPLOYEE CONSOLE

# คู่มือเรียนรู้ฉบับมือใหม่

NestJS · Prisma · OpenAPI · Docker · การทดสอบ · เตรียมตอบคำถาม

|            |                                    |
|------------|------------------------------------|
| ที่มา      | <code>docs/learn/</code> (11 ไฟล์) |
| Commit     | <code>01e23fd</code>               |
| สร้างเมื่อ | 5 ตุลาคม 2026                      |

ฉบับ PDF แปลงจาก Markdown — ถ้าเนื้อหาไม่ตรงกับไฟล์ใน repo ให้ยึดไฟล์ใน repo

# คู่มือเรียนรู้ Employee Console (ฉบับมือใหม่)

คู่มือชุดนี้เขียนให้คนที่ **ยังไม่เคยเขียน NestJS และไม่รู้จัก OpenAPI หรือ Docker** อ่านแล้วเข้าใจว่าโปรเจกต์นี้ทำงานอย่างไร  
ทุกบทจะอธิบายด้วยภาพก่อน แล้วค่อยชี้ไปที่ไฟล์จริงในโปรเจกต์ เปิดโค้ดตามไปได้ทันที

เอกสารเดิมใน `docs/` (architecture, runbook, decisions ฯลฯ) ยังเป็นฉบับอ้างอิงที่ละเอียดที่สุด คู่มือชุดนี้เป็นฉบับ  
อ่านง่ายที่พาไปถึงเอกสารเหล่านั้น ถ้าสองที่พูดไม่ตรงกัน ให้ยึดโค้ดไว้ก่อน

## ลำดับการอ่าน

| # | บท                                                                             | อ่านจบแล้วจะเข้าใจ                                                                                                            |
|---|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1 | ภาพรวมทั้งระบบ ( <code>docs/learn/01-big-picture.md</code> )                   | ชิ้นส่วนแต่ละตัวคืออะไร คอยกันอย่างไร พร้อมตามรอยการ "แก้เงินเดือน Bob" ตั้งแต่กดปุ่มไปจนถึงฐานข้อมูล                         |
| 2 | NestJS สำหรับมือใหม่ ( <code>docs/learn/02-nestjs.md</code> )                  | Module, Controller, Service, Dependency Injection, DTO + ValidationPipe และ exception ผ่านโค้ด <code>employees</code> ของจริง |
| 3 | ฐานข้อมูลและ Prisma ( <code>docs/learn/03-database-prisma.md</code> )          | ตาราง, migration, seed และการแปลงเงิน/วันที่ระหว่าง Prisma กับ API                                                            |
| 4 | OpenAPI และ Swagger UI ( <code>docs/learn/04-openapi.md</code> )               | หน้าเอกสาร API ที่ <code>/api/docs</code> มาจากไหน และ type ของเว็บอยู่ที่ไหน                                                 |
| 5 | Docker ( <code>docs/learn/05-docker.md</code> )                                | Docker ในโปรเจกต์นี้มีแค่ PostgreSQL: image, container, volume, compose                                                       |
| 6 | แนวคิดสำคัญของโปรเจกต์ ( <code>docs/learn/06-key-concepts.md</code> )          | <code>If-Match</code> /version, เงินที่เป็น string, วันที่ที่ไม่มีเวลา, รูปแบบ error และสถานะใน URL                           |
| 7 | การทดสอบ ( <code>docs/learn/07-testing.md</code> )                             | unit, integration และ E2E ต่างกันอย่างไร                                                                                      |
| 8 | แผนที่เอกสารใน <code>docs/</code> ( <code>docs/learn/08-docs-guide.md</code> ) | เอกสารแต่ละไฟล์บอกอะไร ควรอ่านตอนไหน                                                                                          |
| 9 | เตรียมตอบคำถาม ( <code>docs/learn/09-qa-prep.md</code> )                       | คำถามที่น่าจะโดนถามตอนนำเสนอ พร้อมคำตอบที่ชี้ไปที่โค้ด                                                                        |
| — | อภิธานศัพท์ ( <code>docs/learn/glossary.md</code> )                            | คำศัพท์ทั้งหมดในที่เดียว เปิดดูได้เมื่อสงสัย                                                                                  |

ถ้ามีเวลาน้อย ให้อ่านบท 1 → 2 → 6 ก่อน สามบทนี้ครอบคลุมสิ่งที่คนถามบ่อยที่สุด

## สัญลักษณ์ในคู่มือ

- ลองเอง** — คำสั่งที่รันได้จริงบนเครื่อง (ต้อง `pnpm dev:up` และ `pnpm dev` ไว้ก่อน) แต่ละบล็อกมีคำสั่งเดียว
- กับดัก** — จุดที่มือใหม่ (และ AI) พลาดบ่อยในโปรเจกต์นี้
- ชื่อไฟล์โค้ดแสดงเป็น path จาก root ของ repo เช่น `apps/api/src/employees/employees.controller.ts`

# บท 1 – ภาพรวมทั้งระบบ

## โปรเจกต์นี้คืออะไร

Employee Console เป็นเว็บทะเบียนพนักงาน ข้อมูลตั้งต้นมาจากไฟล์ Excel ของโจทก์ (test-exam-data.xlsx (apps/api/prisma/seed-data/test-exam-data.xlsx)) มีพนักงาน 5 คน รหัส 101–105 แต่ละคนมี 7 ฟิลด์ ได้แก่ ID, Name, Department, Salary, Join Date, Status และ Last Updated Date

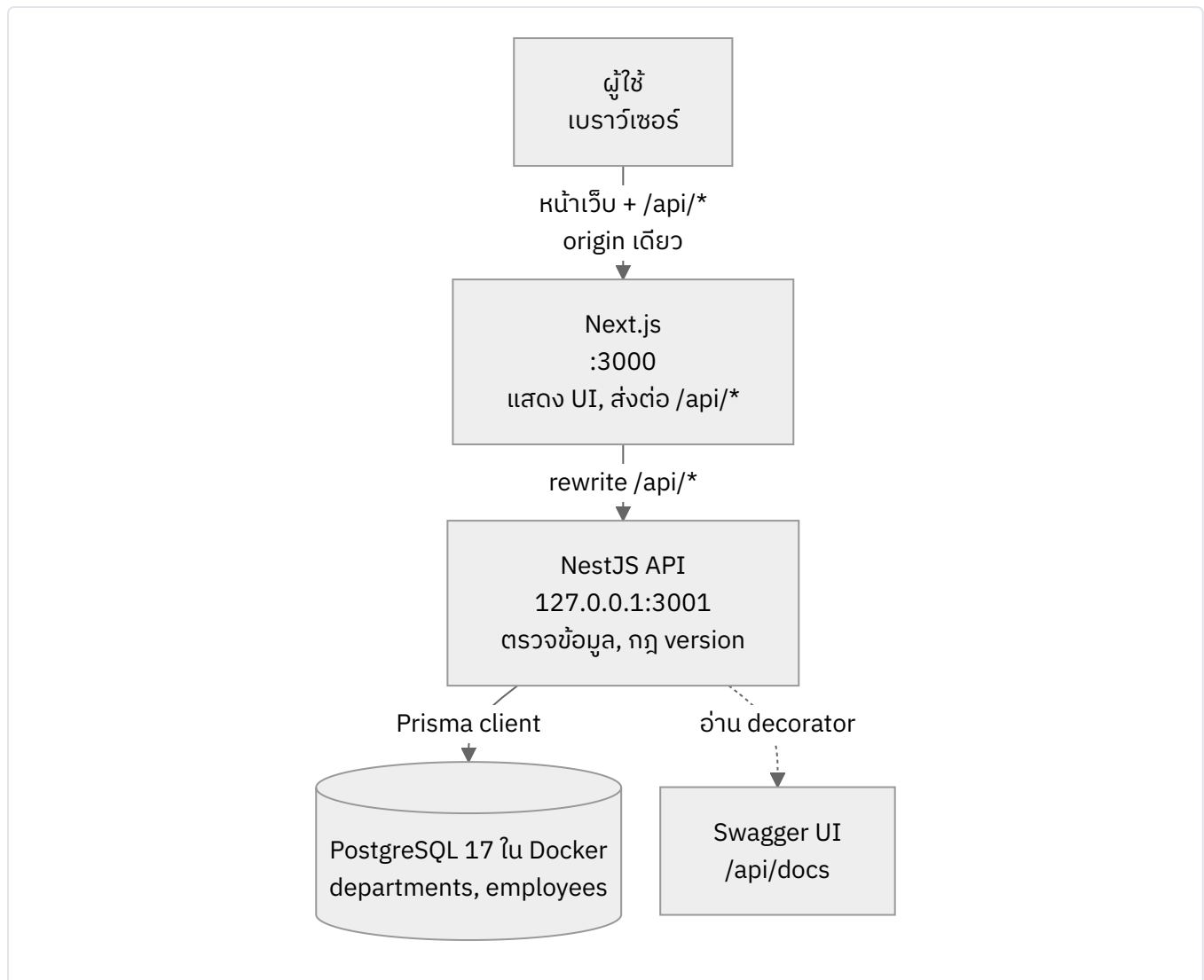
สิ่งที่ผู้ใช้งานทำได้คือ ดูรายการ, ค้นหาชื่อ, กรองตามแผนกหรือสถานะ, เรียงลำดับ, แบ่งหน้า และ เพิ่ม/แก้/ลบ พนักงาน (CRUD) ไม่มีระบบ login ใครเข้าเว็บได้ก็ใช้งานได้ทุกอย่าง ซึ่งเป็นการตัดสินใจโดยตั้งใจ (D-46) และโปรเจกต์ถูกทำให้เล็กลงอีกรอบใน D-56 เหลือแค่สิ่งที่โจทก์ขอ

## เปรียบเทียบกับร้านอาหาร

ถ้ายังนึกภาพไม่ออกว่าแต่ละชิ้นทำอะไร ลองนึกถึงร้านอาหาร

| ในร้านอาหาร                                                              | ในโปรเจกต์                                                                   | อยู่ที่                         |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------|
| ลูกค้า                                                                   | เบราว์เซอร์ของผู้ใช้                                                         | —                               |
| หน้าร้านกับพนักงานเสิร์ฟ: จัดโต๊ะ รับออเดอร์ แล้วเดินไปส่งครัว           | <b>Next.js</b> — แสดงหน้าเว็บ และส่งต่อคำขอ <code>/api/*</code> ไปให้ API    | apps/web                        |
| ครัว: รู้สูตร ตรวจสอบวัตถุดิบ ตัดสินว่าทำได้หรือไม่ แล้ว คุยกับฐานข้อมูล | <b>NestJS</b> — ตรวจสอบข้อมูล ตัดสินว่าบันทึกได้หรือไม่ แล้ว คุยกับฐานข้อมูล | apps/api                        |
| ห้องเก็บของกับสมุดบัญชี                                                  | <b>PostgreSQL</b> — เก็บข้อมูลจริง                                           | Docker container (compose.yaml) |
| เมนูที่เขียนชัดว่าสั่งอะไรได้                                            | <b>Swagger UI</b> — หน้าเอกสาร API ที่ <code>/api/docs</code>                | สร้างจากโค้ดของ API (un 4)      |

ประเด็นสำคัญคือ **พนักงานเสิร์ฟไม่ทำอาหารเอง** Next.js ไม่ต่อฐานข้อมูลและไม่มีกฎธุรกิจ ทุกอย่างที่เป็น "การตัดสินใจ" อยู่ใน NestJS ที่เดียว



- เส้นทึบ = คำขอที่วิ่งตอนใช้งานจริง
- เส้นประ = หน้าเอกสารที่ API สร้างให้จากโค้ดของตัวเอง

ทั้งเว็บและ API รันบนเครื่องเราด้วย `pnpm dev` ส่วน Docker มีหน้าที่เดียวคือรัน PostgreSQL (un 5)

### ทำไมเบราว์เซอร์ไม่คุยกับ API ตรง ๆ

เบราว์เซอร์รู้จักแค่ `http://localhost:3000` เมื่อหน้าเว็บเรียก `fetch('/api/employees')` Next.js จะส่งต่อคำขอนั้นไปที่ NestJS ให้เอง ตามกฎใน `next.config.ts` (`apps/web/next.config.ts`)

```

const apiUrl = process.env.API_INTERNAL_URL ?? 'http://127.0.0.1:3001';
// ...
async rewrites() {
  return [{ source: '/api/:path*', destination: `${apiUrl}/api/:path*` }];
}

```

ผลที่ได้คือ

1. ไม่ต้องเปิด CORS เพราะเบราว์เซอร์คุยกับ origin เดียว
2. API ฟังเฉพาะ `127.0.0.1` (`main.ts` (`apps/api/src/main.ts`)) เครื่องอื่นในเครือข่ายเรียกตรงไม่ได้

## พื้นฐานที่ควรรู้ก่อนอ่านบทต่อไป

### HTTP request และ response

ทุกครั้งที่หน้าเว็บขอข้อมูล จะส่ง **request** ไปหนึ่งก่อน แล้วได้ **response** กลับมาหนึ่งก่อน

```
PATCH /api/employees/104      ← method + path
If-Match: "1"                 ← header (ข้อมูลประกอบ)
Content-Type: application/json

{ "salary": "75000.00" }      ← body (เนื้อหา JSON)
```

```
HTTP/1.1 200 OK                ← status code
Content-Type: application/json

{ "id": 104, "name": "Bob Brown", "salary": "75000.00", ..., "version": 2 }
```

### Method ที่โปรเจกต์ใช้

API นี้ออกแบบแบบ REST คือมองข้อมูลเป็น "resource" ที่มี URL ของตัวเอง แล้วใช้ method บอกว่าจะทำอะไรกับมัน

| Method | ความหมาย    | ตัวอย่างในโปรเจกต์                                                    |
|--------|-------------|-----------------------------------------------------------------------|
| GET    | อ่าน        | GET /api/employees?q=john = ค้นหา, GET /api/employees/104 = ดูคนเดียว |
| POST   | สร้างใหม่   | POST /api/employees                                                   |
| PATCH  | แก้บางฟิลด์ | PATCH /api/employees/104 + header If-Match                            |
| DELETE | ลบ          | DELETE /api/employees/104 + header If-Match                           |

### Status code ที่เจอ

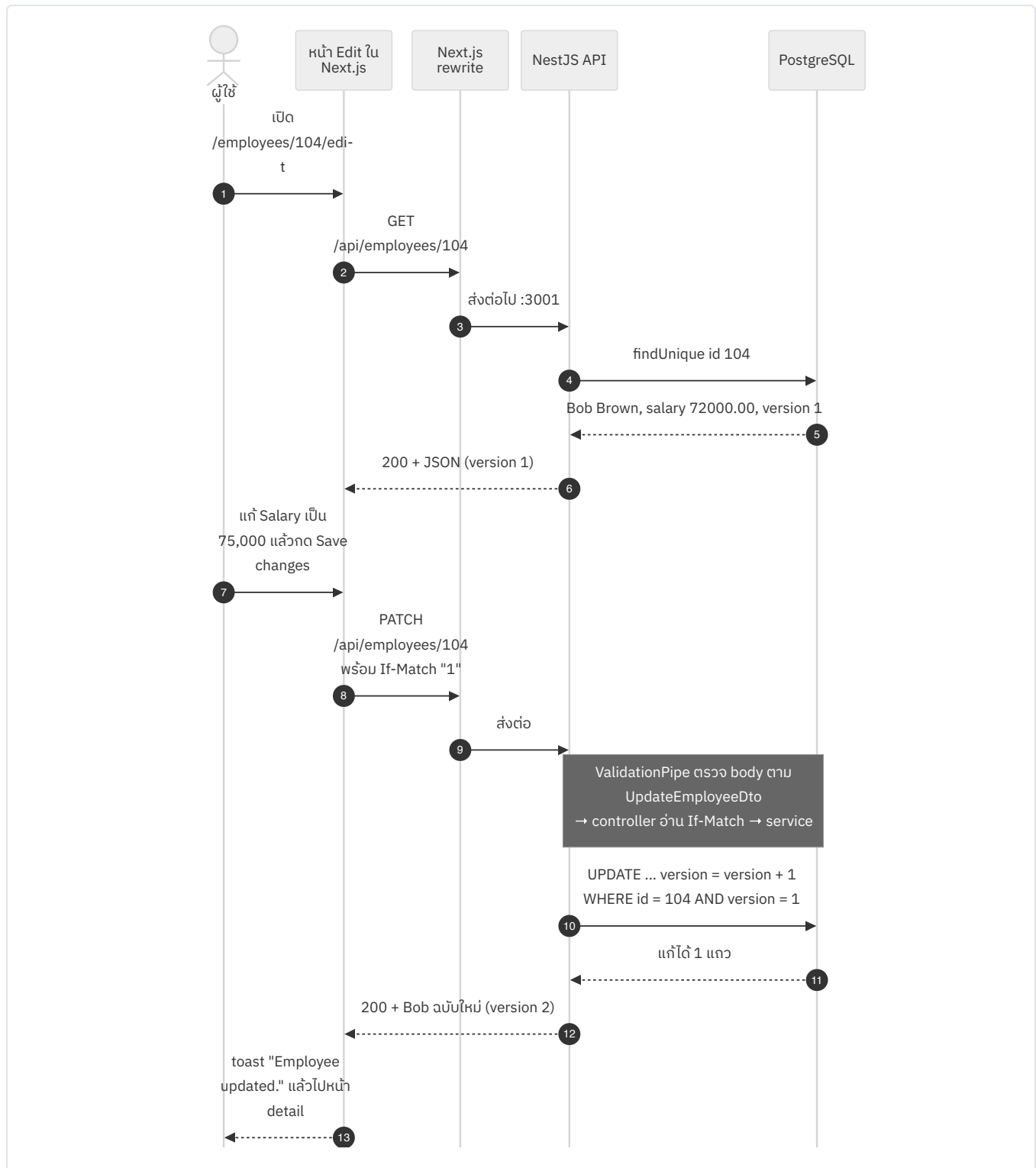
| Code | ความหมาย              | เกิดเมื่อไรในโปรเจกต์                                                              |
|------|-----------------------|------------------------------------------------------------------------------------|
| 200  | สำเร็จ                | อ่านหรือแก้สำเร็จ                                                                  |
| 201  | สร้างสำเร็จ           | POST สำเร็จ                                                                        |
| 204  | สำเร็จแต่ไม่มีเนื้อหา | ลบสำเร็จ                                                                           |
| 400  | ข้อมูลที่ส่งมาผิด     | ชื่อว่าง, salary มีทศนิยม 3 ตำแหน่ง, ส่งฟิลด์ที่ไม่รู้จัก, id ใน path ไม่ใช่ตัวเลข |
| 404  | ไม่พบ                 | ไม่มีพนักงาน ID นี้                                                                |
| 409  | ชนกัน                 | มีคนแก้ record นี้ไปก่อน (version ไม่ตรง)                                          |
| 428  | ขาดเงื่อนไขที่บังคับ  | แก้หรือลบโดยไม่ส่ง If-Match                                                        |

## คำอื่นที่จะเจอ

- **JSON** — รูปแบบข้อมูลที่ส่งไปมา หน้าตาเหมือน object ของ JavaScript
- **TypeScript** — JavaScript ที่มีชนิดข้อมูล (type) ทั้งเว็บและ API เขียนด้วยภาษานี้
- **Decorator** — ป้ายที่แปะไว้บน class หรือ method เช่น `@Get()` เพื่อบอก framework ว่าโค้ดนี้คืออะไร (อธิบายละเอียดในบท 2)
- **Monorepo** — repo เดียวที่มีหลายโปรเจกต์ย่อย ( `apps/api` , `apps/web` , `tests/e2e` ) จัดการด้วย pnpm workspace (`pnpm-workspace.yaml`)

## ตามรอย: แก้เงินเดือน Bob Brown จาก 72,000 เป็น 75,000

ตัวอย่างนี้ร้อยทุกชั้นเข้าด้วยกัน อ่านคู่กับโค้ดจะเห็นภาพชัดที่สุด



| ขั้น  | เกิดอะไรขึ้น                                                                                                                                                        | ไฟล์                                                                                                                                                                                                       |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-2   | หน้า Edit โหลดข้อมูลด้วย hook <code>useEmployee(104)</code>                                                                                                         | <code>edit-employee-view.tsx</code><br>( <code>apps/web/src/app/(console)/employees/[id]/edit/edit-employee-view.tsx</code> ), <code>queries.ts</code> ( <code>apps/web/src/lib/queries.ts</code> )        |
| 3     | Next.js ส่งต่อ <code>/api/*</code> ไปที่ <code>API_INTERNAL_URL</code>                                                                                              | <code>next.config.ts</code> ( <code>apps/web/next.config.ts</code> )                                                                                                                                       |
| 4-6   | Controller แปลง <code>"104"</code> เป็นตัวเลขด้วย <code>parseIntPipe</code> แล้ว service อ่านแถวพร้อมชื่อแผนก และแปลงเงิน/วันที่เป็น string                         | <code>employees.controller.ts</code><br>( <code>apps/api/src/employees/employees.controller.ts</code> ),<br><code>employees.service.ts</code> ( <code>apps/api/src/employees/employees.service.ts</code> ) |
| 7     | ฟอร์มเปลี่ยน <code>75,000</code> เป็น <code>"75000.00"</code> แล้ว <code>changedFields()</code> เทียบกับค่าเดิม ส่งเฉพาะฟิลด์ที่เปลี่ยน คือ <code>{ salary }</code> | <code>edit-employee-view.tsx</code> , <code>salary.ts</code> ( <code>apps/web/src/lib/salary.ts</code> )                                                                                                   |
| 8     | <code>useUpdateEmployee</code> เรียก <code>api()</code> พร้อม header <code>If-Match: "1"</code> (version ของข้อมูลที่แสดงอยู่ในฟอร์ม)                               | <code>queries.ts</code> ( <code>apps/web/src/lib/queries.ts</code> ), <code>api.ts</code><br>( <code>apps/web/src/lib/api.ts</code> )                                                                      |
| 9     | <code>ValidationPipe</code> ตรวจสอบว่า salary เป็นทศนิยมแบบ string ที่ถูกต้อง แล้ว controller อ่าน <code>If-Match</code> เป็นเลข 1                                  | <code>app.ts</code> ( <code>apps/api/src/app.ts</code> ), <code>employee.dto.ts</code><br>( <code>apps/api/src/employees/employee.dto.ts</code> )                                                          |
| 10-11 | Service สั่ง <code>updateMany</code> ที่มีเงื่อนไข <code>{ id: 104, version: 1 }</code> พร้อมเพิ่ม version และตั้ง Last Updated Date เป็น "วันนี้" ตามเวลากรุงเทพฯ  | <code>employees.service.ts</code> ( <code>apps/api/src/employees/employees.service.ts</code> )                                                                                                             |
| 12-13 | API อ่านแถวใหม่ส่งกลับ เว็บเก็บลง cache ของ TanStack Query แสดง toast แล้วเปลี่ยนหน้า                                                                               | <code>edit-employee-view.tsx</code>                                                                                                                                                                        |

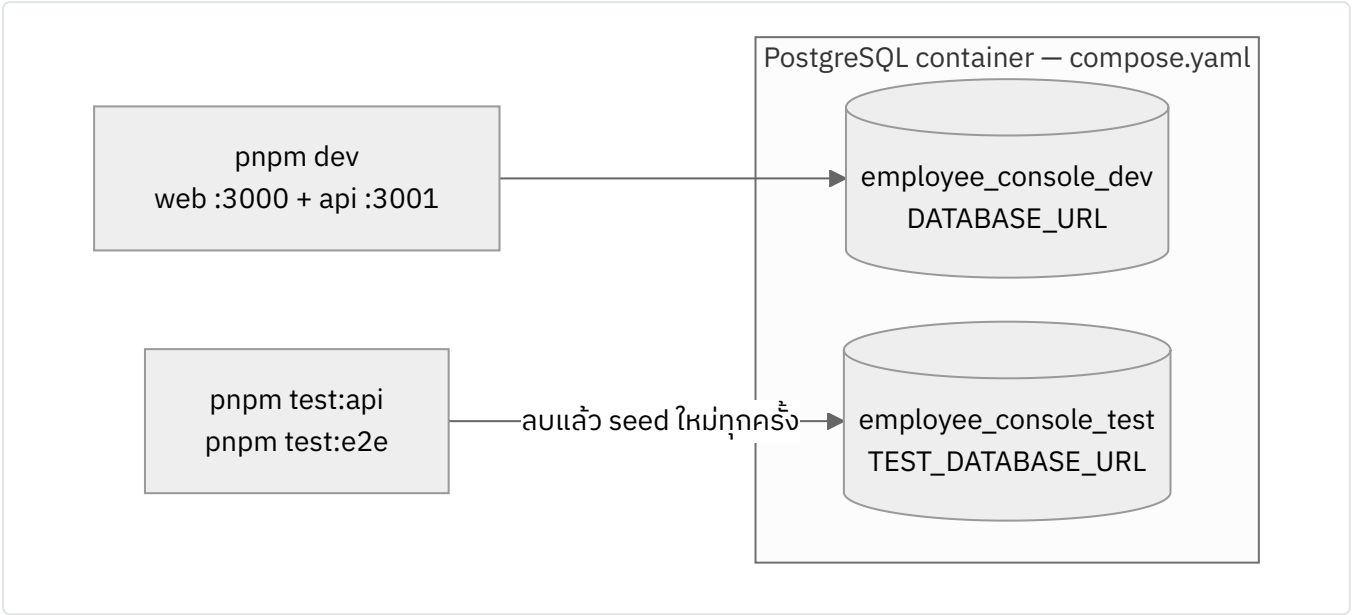
ถ้าระหว่างนั้นมีอีกแท็บเก่า Bob ไปก่อน version ในฐานข้อมูลจะกลายเป็น 2 แล้ว ขั้นที่ 10 จะแก้ได้ 0 แถว service ตรวจสอบว่ายังมี ID นี้อยู่ จึงตอบ **409** แทน (ถ้าไม่มีแล้วจะเป็น 404) หน้าเว็บจะขึ้นแถบเตือน "This employee was changed by another user. Reload the latest version." พร้อมปุ่ม **Reload latest** (รายละเอียดในบท 6)

# โครงสร้าง folder แบบ "อยากทำอะไร ไปที่ไหน"

| อยากแก้                                               | ไปที่                                                                       |
|-------------------------------------------------------|-----------------------------------------------------------------------------|
| หน้าตา ปุ่ม ตาราง ฟอรั่ม                              | apps/web/src/components                                                     |
| route ของหน้าเว็บ ( /employees , /employees/new ฯลฯ)  | apps/web/src/app                                                            |
| สถานะของหน้า list ใน URL (ค้นหา, กรอง, หน้า)          | list-params.ts (apps/web/src/lib/list-params.ts)                            |
| ชนิดข้อมูลที่เว็บรับจาก API และตัวเรียก fetch         | api.ts (apps/web/src/lib/api.ts)                                            |
| endpoint ของ API                                      | employees.controller.ts<br>(apps/api/src/employees/employees.controller.ts) |
| กฎของแต่ละฟิลด์ และของ query (ค้นหา, เรียง, แบ่งหน้า) | employee.dto.ts (apps/api/src/employees/employee.dto.ts)                    |
| การอ่าน/เขียนพนักงานในฐานข้อมูล                       | employees.service.ts (apps/api/src/employees/employees.service.ts)          |
| ตารางในฐานข้อมูล                                      | schema.prisma (apps/api/prisma/schema.prisma) + migration ใหม่              |
| environment variable                                  | .env.example + configuration.md (docs/configuration.md)                     |
| test ผ่านเบราว์เซอร์                                  | tests/e2e/specs                                                             |

## ฐานข้อมูลสองฐาน: dev กับ test

PostgreSQL container ตัวเดียวมีสองฐานข้อมูล แยกกันด้วย URL ใน `.env` ข้อมูลที่ใช้พัฒนาจึงไม่โดน test au



## คำสั่งที่ใช้ทุกวัน

ติดตั้ง dependency ครั้งแรก

```
pnpm install
```

สร้างไฟล์ตั้งค่า `.env` จากตัวอย่าง (ครั้งเดียว)

```
cp .env.example .env
```

เปิดฐานข้อมูล รองนพร้อม แล้ว migrate + seed

```
pnpm dev:up
```

รันเว็บกับ API แบบ hot reload แล้วเปิด <http://localhost:3000>

```
pnpm dev
```

ปิด container (ข้อมูลยังอยู่ใน volume)

```
pnpm down
```

**กับดัก:** `pnpm dev:up` seed ก็ต่อเมื่อตาราง employees ว่างเท่านั้น ถ้าเคยแก้หรือลบข้อมูลไปแล้วและอยากได้ 5 records เดิมคืน ให้ใช้ `pnpm db:reset` (ลบพนักงานทั้งหมดแล้ว seed ใหม่)

**กับดัก:** `.env` ไม่อยู่ใน git (ดู `.gitignore`) อย่า commit และถ้าลืม `cp .env.example .env` คำสั่ง migrate และ test จะหา `DATABASE_URL` / `TEST_DATABASE_URL` ไม่เจอ

---

ต่อไป: บท 2 – NestJS สำหรับมือใหม่ (docs/learn/02-nestjs.md)

# บท 2 – NestJS สำหรับมือใหม่

บทนี้สอน NestJS ผ่านโค้ดจริงของโปรเจกต์ ทั้ง API มีราวสิบไฟล์ใน apps/api/src (ไม่นับ generated/ ที่ Prisma สร้างให้) อ่านจบบทนี้แล้วเปิดอ่านได้ครบทุกไฟล์

## NestJS คืออะไร

NestJS เป็น framework สำหรับเขียน backend ด้วย TypeScript ข้างในยังใช้ **Express** (ตัวรับ HTTP ที่นิยมที่สุดของ Node.js) แต่ Nest เพิ่ม "โครง" ให้ว่าโค้ดแต่ละประเภทต้องอยู่ที่ไหน

ถ้าเขียน Express เปล่า ๆ ทุกอย่างมักไปกองรวมในฟังก์ชันเดียว ทั้งรับ request, ตรวจสอบข้อมูล, คุยฐานข้อมูล และจัดการ error พอโปรเจกต์โตก็หาอะไรไม่เจอ Nest จึงแยกงานเป็นชั้น ๆ และให้แต่ละชั้นมีหน้าที่เดียว

## คำศัพท์ที่ต้องรู้

| คำ                           | เปรียบเทียบ                                              | ในโปรเจกต์นี้                                                               |
|------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------|
| Module                       | แผนกในบริษัท รวมคนที่ทำงานเรื่องเดียวกัน                 | AppModule , PrismaModule , EmployeesModule                                  |
| Controller                   | พนักงานต้อนรับ รับเรื่อง แล้วส่งต่อให้คนที่ทำจริง        | EmployeesController รับ GET/POST/PATCH/DELETE                               |
| Service (Provider ชนิดหนึ่ง) | ผู้เชี่ยวชาญที่ลงมือทำงานจริง                            | EmployeesService คุยกับฐานข้อมูลและดึงกฎเรื่อง version                      |
| Dependency Injection (DI)    | ฝ่ายบุคคลจัดคนมาให้ ไม่ต้องไปจ้างเอง                     | Nest สร้าง PrismaService แล้วส่งให้ EmployeesService อัตโนมัติ              |
| Pipe                         | ด่านตรวจสอบเอกสาร ตรวจสอบและแปลงข้อมูล ก่อนถึงมือคนทำงาน | ValidationPipe ตรวจสอบ body/query ตาม DTO, ParseIntPipe แปลง :id เป็นตัวเลข |
| DTO                          | แบบฟอร์มที่บอกว่าต้องกรอกช่องไหนแบบไหน                   | CreateEmployeeDto , UpdateEmployeeDto , ListEmployeesQuery                  |
| Exception                    | ใบแจ้งว่าทำไม่ได้ พร้อมเหตุผล                            | NotFoundException (404), ConflictException (409)                            |

**Provider** คือของทุกอย่างที่ Nest สร้างแล้วแจกจ่ายผ่าน DI ได้ Service เป็น provider ที่พบบ่อยที่สุด

Nest ยังมีจุดเสียบอื่นอีก เช่น Middleware, Guard, Interceptor และ Exception Filter แต่โปรเจกต์นี้ไม่ได้ใช้ เพราะไม่มี login, ไม่ห่อ response และใช้รูปแบบ error มาตรฐานของ Nest ตรง ๆ (D-56)

## Decorator คืออะไร

Decorator คือ **ป้ายที่ขึ้นต้นด้วย @** แปะไว้บน class, method, parameter หรือ property เพื่อบอก Nest ว่าโค้ดนั้นมีหน้าที่อะไร ตัว decorator เองไม่ได้ทำงาน แต่ Nest จะอ่านป้ายเหล่านี้ตอนเริ่มโปรแกรม แล้วต่อสายทุกอย่างให้

ดูส่วนที่สั้นที่สุดของ `employees.controller.ts` (`apps/api/src/employees/employees.controller.ts`)

```
@ApiTags('employees') // ① ป้ายสำหรับ Swagger: จัดกลุ่มว่า "employees"
@Controller('employees') // ② class นี้ดูแล path /employees
export class EmployeesController {
  constructor(private readonly employees: EmployeesService) {} // ③ ขอ EmployeesService ผ่าน DI

  @Get('/:id') // ④ method นี้ตอบ GET /employees/:id
  get(@Param('id', ParseIntPipe) id: number) { // ⑤ อ่าน :id แล้วแปลง "104" → 104
    return this.employees.get(id); // ⑥ ค่าที่ return กลายเป็น JSON ให้เอง
  }
}
```

- ② + ④ รวมกันเป็น `/employees/:id` และเพราะ `app.ts` (`apps/api/src/app.ts`) ตั้ง `app.setGlobalPrefix('api')` path จริงจึงเป็น `GET /api/employees/:id`
- ③ คือ DI: เราไม่ได้เขียน `new EmployeesService()` เอง Nest ส่งมาให้
- ⑤ ถ้า `id` ไม่ใช่ตัวเลข เช่น `/api/employees/abc` `ParseIntPipe` จะตอบ 400 ก่อนถึง method
- ① ไม่มีผลกับการทำงาน มีไว้ให้หน้า Swagger UI (un 4)

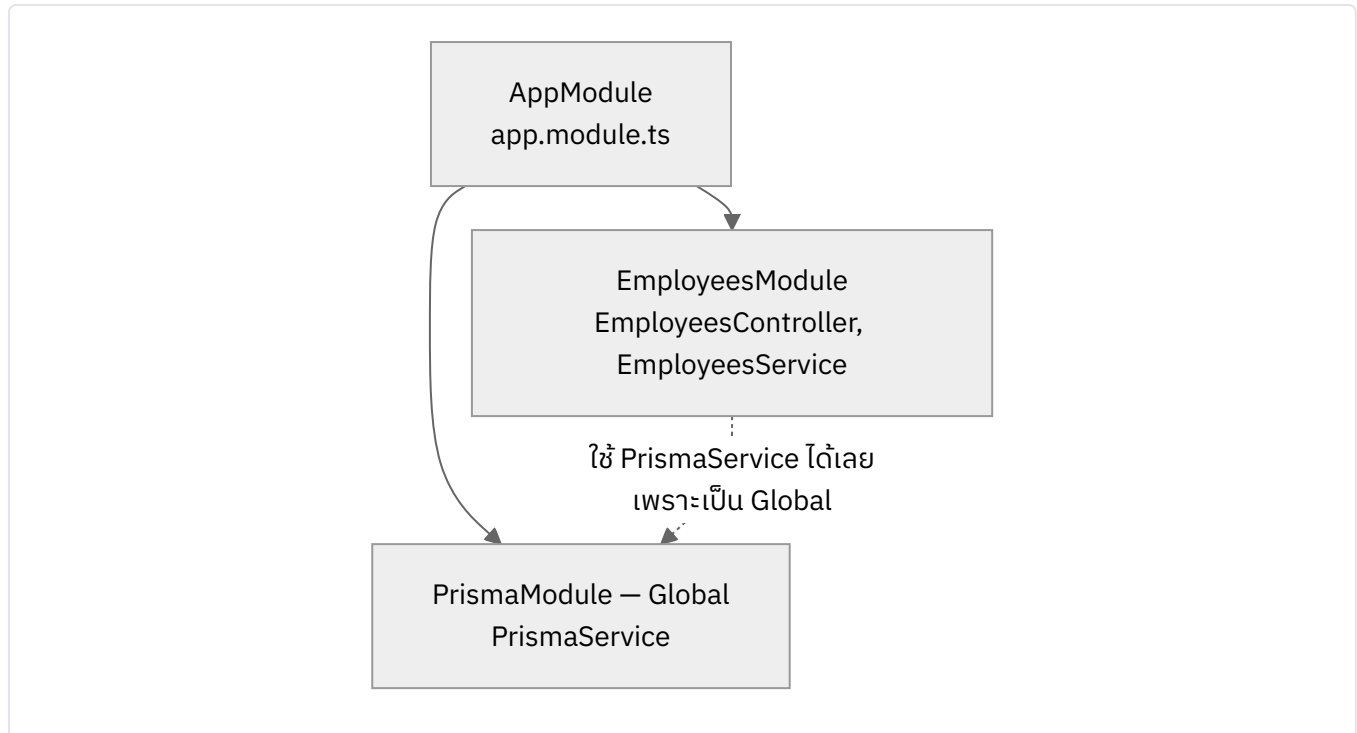
Decorator ที่จะเจอใน controller นี้

| Decorator                               | อ่านอะไรจาก request                                    | ตัวอย่าง                                                                   |
|-----------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------|
| <code>@Param('id', ParseIntPipe)</code> | ค่าใน path แล้วแปลงเป็นตัวเลข                          | <code>/api/employees/104</code> → <code>104</code>                         |
| <code>@Query()</code>                   | query string ทั้งหมด → <code>ListEmployeesQuery</code> | <code>?q=john&amp;page=2</code> → <code>{ q: 'john', page: 2, ... }</code> |
| <code>@Body()</code>                    | JSON body → DTO                                        | <code>{ "salary": "75000.00" }</code>                                      |
| <code>@Headers('if-match')</code>       | header หนึ่งตัว                                        | <code>"1"</code>                                                           |
| <code>@HttpCode(204)</code>             | (ไม่ได้อ่าน) ตั้ง status ของคำตอบ                      | <code>DELETE</code> ตอบ 204 แทน 200                                        |

**กับดัก:** ค่าจาก URL เป็น string เสมอ ที่ `page` กลายเป็นตัวเลขได้เพราะ `ValidationPipe` ตั้ง `transform: true` และ DTO ใส่ `@Type(() => Number)` ไว้ ถ้าเพิ่ม query ที่เป็นตัวเลขแล้วลืม `@Type` จะได้ 400 เพราะ `@IsInt()` เจอ string

## Module — กล่องที่รวมของที่เกี่ยวข้องกัน

ทุก controller และ service ต้องอยู่ใน module ใด module หนึ่ง และ module ทั้งหมดถูกรวมไว้ที่ `AppModule` ซึ่งเป็นรากของต้นไม้ โปรเจกต์นี้มีแค่สามกล่อง



`app.module.ts` (`apps/api/src/app.module.ts`) กับ `employees.module.ts` (`apps/api/src/employees/employees.module.ts`)

```
@Module({
  imports: [PrismaModule, EmployeesModule], // รวม module ทั้งหมดของแอป
})
export class AppModule {}

@Module({
  controllers: [EmployeesController], // controller ของ module นี้
  providers: [EmployeesService], // service ที่ Nest จะสร้างและแจกจ่าย
})
export class EmployeesModule {}
```

- **providers** — ของที่ Nest สร้างให้ module นี้ใช้
- **exports** — ของที่ยอมให้ module อื่นใช้ `prisma.module.ts` (`apps/api/src/prisma/prisma.module.ts`) export `PrismaService`
- **@Global()** — `PrismaModule` ประกาศเป็น global ทุก module จึงใช้ `PrismaService` ได้โดยไม่ต้องใส่ใน `imports` (เพราะ feature ไหนก็ต้องคุยกับฐานข้อมูล)

กฎของโปรเจกต์ (จาก `apps/api/AGENTS.md`): feature ใหม่ = folder ของตัวเองที่มี `<name>.module.ts` + controller + service + dto แล้วเพิ่มเข้า `imports` ของ `AppModule`

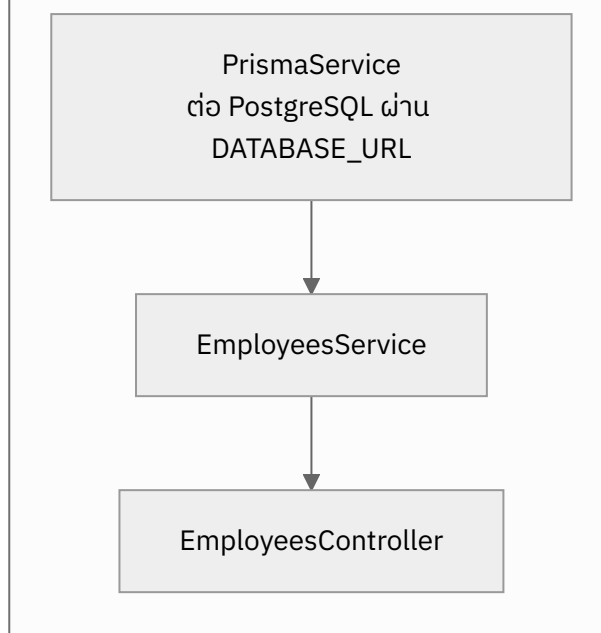
## Dependency Injection – ไม่ต้อง new เอง

ดู constructor ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`)

```
@Injectable() // บอก Nest ว่า class นี้ให้ DI สร้างได้
export class EmployeesService {
  constructor(private readonly prisma: PrismaService) {} // ขอดด้วย "ชนิด" ของ class
}
```

Service ไม่รู้เลยว่า `PrismaService` ถูกสร้างอย่างไร แค่ประกาศว่า "ต้องใช้" ตอนแอปเริ่ม Nest จะไล่ดูว่าใครต้องใช้อะไร แล้วสร้างให้ตามลำดับ

Nest DI container — สร้างครั้งเดียวแล้วใช้ร่วมกัน



ลูกศร  $A \rightarrow B$  หมายถึง "B ต้องใช้ A"

ประโยชน์ที่เห็นในโปรเจกต์นี้

- `PrismaService` มีตัวเดียวทั้งแอป ทุกคำขอใช้ connection pool ชุดเดียวกัน
- Nest เรียก `onModuleDestroy()` ของ `prisma.service.ts` (`apps/api/src/prisma/prisma.service.ts`) ให้ตอนแอปหยุด (เปิดไว้ด้วย `app.enableShutdownHooks()`) การเชื่อมต่อจึงถูกปิดเรียบร้อย
- โค้ดที่อยู่นอก Nest เช่น script seed และ test ใช้ `createPrismaClient()` จากไฟล์เดียวกัน จึงต่อฐานข้อมูลแบบเดียวกัน

**กับดัก:** Nest รู้ว่า constructor ต้องการ `PrismaService` จาก "decorator metadata" ที่ TypeScript compiler ฝังไว้ esbuild (ซึ่ง `tsx` ใช้) ไม่สร้างข้อมูลนี้ โค้ดที่สร้างแอป Nest จึงต้อง compile ด้วย `tsc` (`nest start`, `pnpm build`) หรือ SWC (Vitest ใช้ `unplugin-swc`) ถ้าใช้ esbuild DI จะพังแบบเจียบ ๆ (D-13) ส่วน `pnpm db:seed` รันด้วย `tsx` ได้เพราะไม่ได้สร้างแอป Nest

## แอปเริ่มทำงานอย่างไร

`main.ts` (`apps/api/src/main.ts`) มีแค่สามบรรทัด

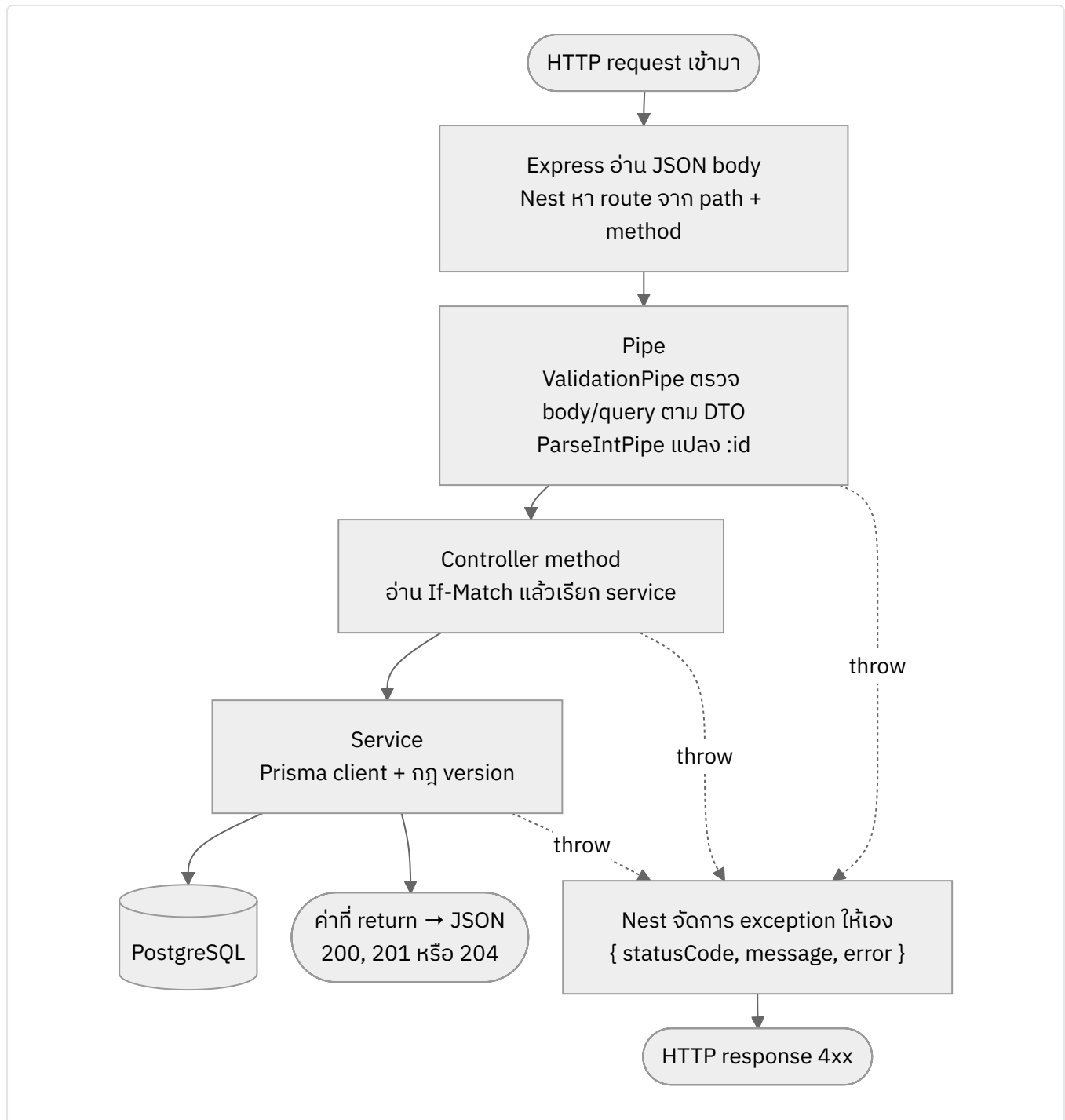
```
loadEnv(); // อ่าน .env ที่ root ของ repo
const app = await createApp(); // ประกอบแอป
await app.listen(Number(process.env.PORT ?? 3001), '127.0.0.1');
```

- `env.ts` (`apps/api/src/env.ts`) ใช้ `process.loadEnvFile` ของ Node 24 อ่าน `.env` ตัวแปรที่ตั้งไว้ก่อนแล้ว (เช่น จาก `test`) จะไม่ถูกทับ
- `app.ts` (`apps/api/src/app.ts`) มี `createApp()` ที่ใช้ร่วมกันสองที่ คือตอนรันจริงกับตอน `integration test` แอปจึงหน้าตาเดียวกัน

```
app.setGlobalPrefix('api'); // ทุก route ขึ้นต้นด้วย /api
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, openApi)); // หน้า /api/docs
app.enableShutdownHooks(); // ปิด Prisma ให้ตอนแอปหยุด
```

## ทางเดินของ request หนึ่งคำขอ

นี่คือภาพที่สำคัญที่สุดของบทนี้



ถ้าด้านไหน throw exception คำขอจะหยุดตรงนั้น และ Nest แปลง exception เป็น JSON ให้เอง ไม่มีข้อความ SQL หรือ stack trace หลุดออกไป

## Validation: DTO + ValidationPipe

**DTO (Data Transfer Object)** คือ class ที่บอกว่า body หรือ query ต้องมีหน้าตาแบบไหน กฎเขียนด้วย decorator ของ library `class-validator` ดู `employee.dto.ts` (`apps/api/src/employees/employee.dto.ts`)

```
/** A decimal string, never a JSON number, so no floating-point rounding can happen on the way.
 */
@ApiProperty({ example: '62000.00', description: 'Decimal string, at most 2 decimal places' })
// สำหรับ Swagger
@IsString({ message: 'Salary must be sent as a string, e.g. "65000.00".' })
// ต้องเป็น string
@Matches(/^\\d{1,10}(\\.\\d{1,2})?$/, { message: 'Salary must be a positive number with at most 2
decimal places.' })
salary: string;
```

DTO สามตัวในไฟล์นี้

| DTO                | ใช้กับ                                       | จุดที่ควรรู้                                                                                                                                                |
|--------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CreateEmployeeDto  | body ของ POST                                | 5 ฟیلด์ที่ผู้ใช้กรอกได้ ชื่อถูก trim และ normalize ด้วย <code>@Transform</code> ก่อนตรวจ                                                                    |
| UpdateEmployeeDto  | body ของ PATCH                               | <code>PartialType(CreateEmployeeDto, { skipNullProperties: false })</code> =<br>ทุกฟیلด์ไม่บังคับ แต่ถ้าส่ง <code>null</code> มาจะยังถูกตรวจ (และถูกปฏิเสธ) |
| ListEmployeesQuery | query ของ GET<br><code>/api/employees</code> | ค่า default: <code>page</code> 1, <code>pageSize</code> 20, <code>sortBy</code> id, <code>sortOrder</code> asc, <code>status</code> all                     |

ValidationPipe ตั้งค่าไว้สามอย่างใน `app.ts` (`apps/api/src/app.ts`)

- `whitelist` + `forbidNonWhitelisted` — ส่งฟیلด์ที่ DTO ไม่มีมา เช่น `id`, `version`, `lastUpdatedDate` หรือ `nickname` จะได้ 400 พร้อมข้อความ `property id should not exist` ผู้ใช้จึงกำหนด ID หรือ version เองไม่ได้
- `transform` — แปลง JSON/query ให้เป็น instance ของ DTO พร้อมค่า default ส่วน body ไม่ได้แปลงชนิดให้ ส่ง `"true"` (string) มาที่ `isActive` จึงได้ 400

## เดินผ่านโค้ด: PATCH /api/employees/:id

### Controller — แทะข้อมูลจาก request แล้วส่งต่อ

จาก employees.controller.ts (apps/api/src/employees/employees.controller.ts)

```
/** `If-Match: "3"` → 3. Edits and deletes must say which version of the record they are based on. */
function parseIfMatch(header: string | undefined): number {
  if (!header) throw new HttpException('Send If-Match with the employee version you edited.', HttpStatus.PRECONDITION_REQUIRED); // 428
  const version = Number(header.replaceAll('"', '')); // "3" หรือ 3 ก็ได้ผลเดียวกัน
  if (!Number.isInteger(version)) throw new BadRequestException('If-Match must be the employee version, e.g. "3".'); // 400
  return version;
}

@Patch('/:id')
@ifMatchHeader // ป้าย Swagger: บังคับ header If-Match
update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateEmployeeDto, @Headers('if-match') ifMatch?: string) {
  return this.employees.update(id, dto, parseIfMatch(ifMatch));
}
```

สังเกตว่า controller **ไม่แตะฐานข้อมูล** ทำแค่แปลง HTTP ให้เป็นค่าที่ใช้ได้ แล้วโยนให้ service

### Service — ฐานข้อมูลกับกฎ version

จาก employees.service.ts (apps/api/src/employees/employees.service.ts)

```
async update(id: number, dto: UpdateEmployeeDto, version: number): Promise<Employee> {
  const { count } = await this.prisma.employee.updateMany({
    where: { id, version }, // แก้เฉพาะเมื่อ version ในฐานยังตรงกับที่ client เห็น
    data: {
      ...dto,
      joinDate: dto.joinDate ? fromDateOnly(dto.joinDate) : undefined,
      lastUpdatedDate: todayInBangkok(), // วันนี้ตามเวลากรุงเทพ
      version: { increment: 1 }, // version + 1
    },
  });
  if (count === 0) await this.throwNotFoundOrConflict(id); // ไม่โดนสักราว → 404 หรือ 409
  return this.get(id); // อ่านแถวใหม่ส่งกลับ
}

private async throwNotFoundOrConflict(id: number): Promise<never> {
  const exists = await this.prisma.employee.count({ where: { id } });
  if (!exists) throw new NotFoundException('This employee does not exist or was deleted.');
```

สามจุดที่ควรจำ

1. **ตรวจสอบ version กับเขียนในคำสั่งเดียว** — `updateMany` ที่มี `where: { id, version }` กลายเป็น `UPDATE ... WHERE id = $1 AND version = $2` ถ้ามีสองคำขอพร้อมกัน PostgreSQL ให้ผ่านได้แค่คำขอเดียว อีกคำขอได้ 0 แถว (มี integration test ยิ่งพร้อมกัน 3 คำขอ ได้ 200 ครั้ง 409 สองครั้ง)
2. **count 0 แยกได้สองกรณี** — ไม่มี ID นี้แล้ว (404) หรือมีแต่ version เปลี่ยนไปแล้ว (409)
3. **ทุกการแก้ไขเพิ่ม version เสมอ** — API ไม่ได้เทียบว่าค่าเปลี่ยนจริงไหม หน้าเว็บเป็นฝ่ายกันไว้ ถ้าไม่ได้ทำอะไรจะไม่ส่ง PATCH เลย (un 6)

`remove()` ใช้หลักเดียวกันด้วย `deleteMany({ where: { id, version } })`

## Error — ใช้ exception ที่ Nest มีให้

| โยนอะไร                                | Status | body ที่ได้                                                                                                                                                                   |
|----------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>NotFoundException('...')</code>  | 404    | <code>{ "message": "This employee does not exist or was deleted.", "error": "Not Found", "statusCode": 404 }</code>                                                           |
| <code>ConflictException('...')</code>  | 409    | <code>{ "message": "This employee was changed by another user. Reload the latest version.", "error": "Conflict", "statusCode": 409 }</code>                                   |
| <code>HttpException('...', 428)</code> | 428    | <code>{ "statusCode": 428, "message": "Send If-Match with the employee version you edited." }</code> (ไม่มี <code>error</code> เพราะสร้างจาก <code>HttpException</code> ตรงๆ) |
| <code>ValidationPipe</code>            | 400    | <code>{ "message": ["...", "..."], "error": "Bad Request", "statusCode": 400 }</code> ( <code>message</code> เป็น array)                                                      |

## ลองเอง

รัน `pnpm dev` ไว้ก่อน แล้วเปิด terminal อีกหน้าต่าง คำสั่งด้านล่างยิงตรงไปที่ API ใช้ `127.0.0.1` เพราะ API ฟังเฉพาะที่อยู่นี้ ทุกคำสั่ง **ไม่เปลี่ยนข้อมูล**

1. **ค้นหาชื่อ** — ต้องใส่ URL ในเครื่องหมายคำพูด เพราะ zsh ตีความ `?` เป็น wildcard

```
curl -s 'http://127.0.0.1:3001/api/employees?q=john'
```

2. **ดูพนักงานคนเดียว** — สังเกต `"version":1` ใน body

```
curl -i http://127.0.0.1:3001/api/employees/104
```

3. **แก้ไขไม่ส่ง If-Match** — ได้ 428

```
curl -i -X PATCH http://127.0.0.1:3001/api/employees/104 -H 'Content-Type: application/json' -d '{"salary": "75000.00"}'
```

4. **แก้ไขส่ง version ผิด** — ได้ 409

```
curl -i -X PATCH http://127.0.0.1:3001/api/employees/104 -H 'Content-Type: application/json' -H 'If-Match: "99"' -d '{"salary": "75000.00"}'
```

5. **ส่ง salary เป็นตัวเลขแทน string** — ได้ 400 พร้อมข้อความสองข้อใน `message`

```
curl -s -X PATCH http://127.0.0.1:3001/api/employees/104 -H 'Content-Type: application/json' -H 'If-Match: "1"' -d '{"salary": 75000}'
```

## 6. id ที่ไม่ใช่ตัวเลข — ได้ 400 จาก `ParseIntPipe` (ไม่ใช่ 404)

```
curl -s http://127.0.0.1:3001/api/employees/abc
```

ก๊อปดัก: ใน `curl -i` จะเห็น header `ETag: W/"c1-..."` อันนี้ Express สร้างให้อัตโนมัติจากเนื้อหา response **ไม่ใช่** เลข **version** ของโปรเจกต์ ถ้าเอาไปใส่ `If-Match` จะได้ 400 ให้ใช้ค่า `version` ใน body เสมอ

ถ้าลองแก้จริงด้วย `version` ที่ถูกต้องแล้วอยากคืนข้อมูลเป็น 5 records เดิม

```
pnpm db:reset
```

## สูตร: เพิ่มของใหม่ทีละชั้น

### เพิ่มตัวกรอง Join Date ช่วงเริ่ม-จบ (โจทย์ข้อ Live Coding)

ทำจากชั้นในสุดออกมา

1. **DTO** — เพิ่ม `joinDateFrom?` / `joinDateTo?` ใน `ListEmployeesQuery` พร้อม `@IsOptional()`, `@Matches(/^\\d{4}-\\d{2}-\\d{2}$/)` และ `@ApiPropertyOptional()`
2. **Service** — เพิ่มเงื่อนไขใน `where` ของ `list()` เช่น `joinDate: { gte: fromDateOnly(from), lte: fromDateOnly(to) }` (ใช้ `fromDateOnly` เพื่อไม่ให้วันเลื่อน)
3. **Web** — เพิ่มฟิลด์ใน `ListParams`, `DEFAULT_PARAMS` และ `readListParams()` ของ `list-params.ts` (`apps/web/src/lib/list-params.ts`) แล้วเพิ่มช่องใน `employee-filters.tsx` (`apps/web/src/components/employees/employee-filters.tsx`) — `toSearch()` จะส่งค่าไปที่ URL และ API ให้เอง
4. **Test** — integration test ใน `employees.test.ts` (`apps/api/test/integration/employees.test.ts`) และถ้าแก้หน้าจอก็เพิ่ม E2E

### เพิ่ม feature module ใหม่

1. สร้าง folder `apps/api/src/<name>/` มี `<name>.module.ts`, `<name>.controller.ts`, `<name>.service.ts`, `<name>.dto.ts`
2. ใส่ `@Controller('<name>')` (prefix `api` จะต่อให้เอง) และ `@Injectable()` ให้ service
3. ลงทะเบียน controller และ provider ใน module แล้วเพิ่ม module เข้า `imports` ของ `AppModule`
4. โยน error ด้วย exception ของ Nest เช่น `NotFoundException`
5. ฝั่งเว็บเพิ่ม type ใน `lib/api.ts` และ hook ใน `lib/queries.ts`
6. เขียน integration test ใน `apps/api/test/integration`

กับดักที่เจอบ่อยในโปรเจกต์นี้

| กับดัก                                                                                                       | ทำไม                                                                            | ที่มา                                |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------|
| import ต้องลงท้าย <code>.js</code> แม้ไฟล์จริงเป็น <code>.ts</code> เช่น <code>./employees.service.js</code> | โปรเจกต์เป็น ESM แบบ <code>nodenext</code>                                      | apps/api/AGENTS.md                   |
| ห้ามรันโค้ดที่สร้างแอป Nest ด้วย <code>tsx/esbuild</code>                                                    | esbuild ไม่สร้าง decorator metadata ผลคือ DI พังแบบเจียบ ๆ                      | D-13, ai-usage.md (docs/ai-usage.md) |
| อย่าเปลี่ยน <code>updateMany / deleteMany</code> เป็น "อ่านก่อนแล้วค่อยเขียน"                                | ระหว่างอ่านกับเขียนอาจมีคนแก้แทรก ทำให้กันการเขียนกับไม่ได้                     | apps/api/AGENTS.md                   |
| อย่าลบบรรทัด escape <code>%</code> กับ <code>_</code> ใน <code>list()</code>                                 | Prisma <code>contains</code> ไม่ escape ให้ คั่น <code>%</code> แล้วจะได้ทุกแถว | un 3                                 |
| เพิ่ม env ใหม่ต้องแก้ <code>.env.example</code> และ <code>configuration.md</code> (docs/configuration.md)    | โค้ดอ่าน <code>process.env</code> ตรง ๆ ไม่มีจุดตรวจกลาง                        | apps/api/AGENTS.md                   |

ต่อไป: un 3 – ฐานข้อมูลและ Prisma (docs/learn/03-database-prisma.md)

# บท 3 – ฐานข้อมูลและ Prisma

## PostgreSQL อยู่ที่ไหน

PostgreSQL 17 รันอยู่ใน Docker container (un 5) เปิดด้วย `pnpm dev:up` แล้วรับการเชื่อมต่อที่ `127.0.0.1:5432` เท่านั้น เครื่องอื่นใน LAN ต่อไม่ได้

ใน container เดียวมีสองฐานข้อมูล

| ฐานข้อมูล                          | ใช้กับ                                                                                             | ถูกสร้างเมื่อ                                                                         |
|------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>employee_console_dev</code>  | <code>pnpm dev</code> (ตัวแปร <code>DATABASE_URL</code> )                                          | container เปิดครั้งแรก ( <code>POSTGRES_DB</code> ใน <code>compose.yaml</code> )      |
| <code>employee_console_test</code> | <code>pnpm test:api</code> และ <code>pnpm test:e2e</code> (ตัวแปร <code>TEST_DATABASE_URL</code> ) | test รัน <code>prisma migrate deploy</code> ครั้งแรก<br>Prisma สร้างฐานให้ถ้ายังไม่มี |

ฐาน test ถูกลบข้อมูลแล้ว seed ใหม่ทุกครั้งที่รัน test ห้ามตั้ง `TEST_DATABASE_URL` ให้ใช้ฐานเดียวกับ `DATABASE_URL` ซึ่ง `setup.ts` (`apps/api/test/integration/setup.ts`) ตรวจสอบและปฏิเสธให้

## ตารางมีอะไรบ้าง

| DEPARTMENTS |            |    |                                   |
|-------------|------------|----|-----------------------------------|
| varchar     | id         | PK | engineering, marketing, sales, hr |
| varchar     | name       | UK | Engineering ...                   |
| int         | sort_order |    | ลำดับจากข้อมูล Excel              |

มีพนักงาน

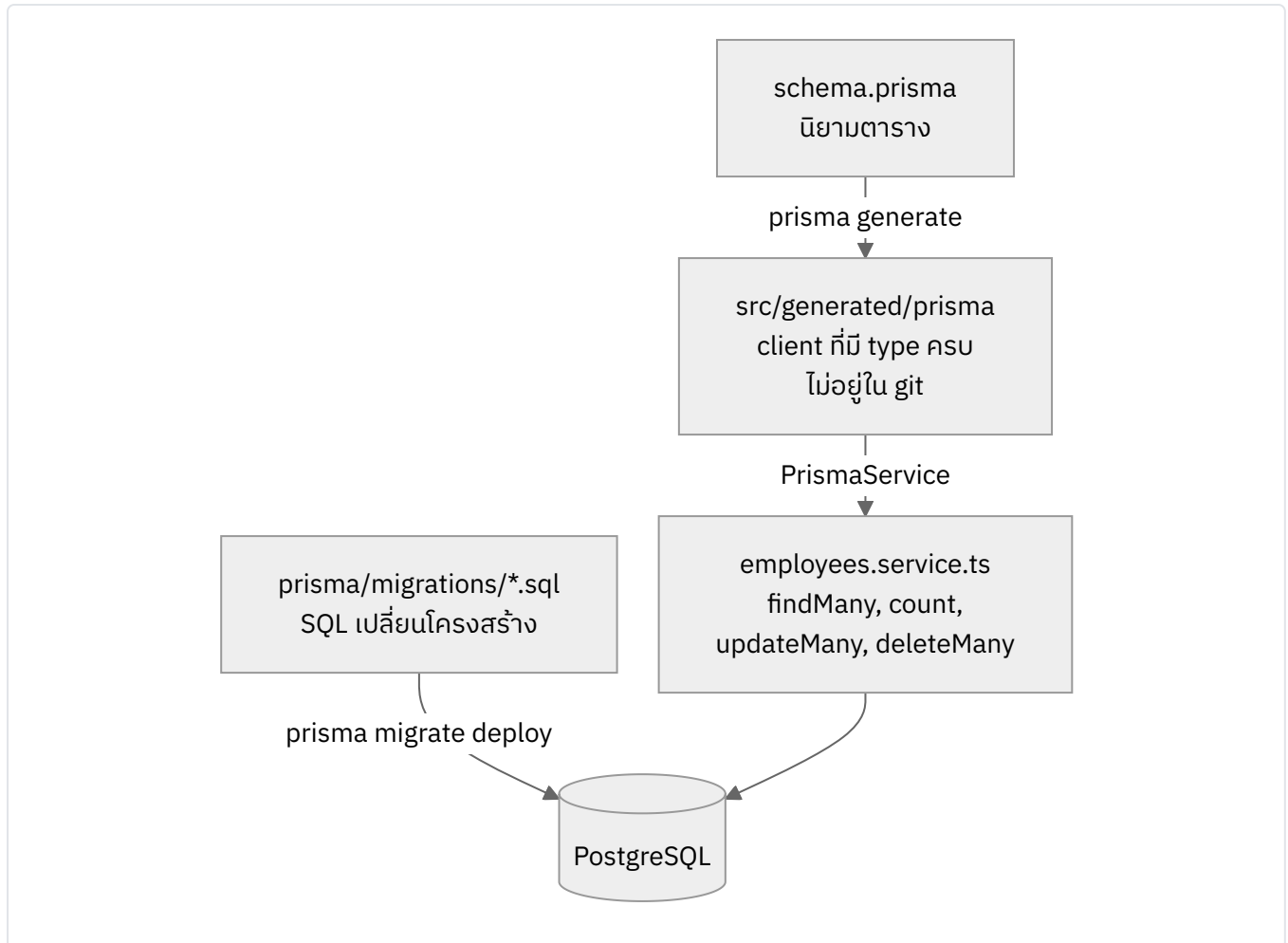
| EMPLOYEES  |                   |    |                                         |
|------------|-------------------|----|-----------------------------------------|
| int        | id                | PK | identity, seed 101-105, คนใหม่เริ่ม 106 |
| varchar    | name              |    | 1-100 ตัวอักษร                          |
| varchar    | department_id     | FK | ลบแผนกที่มีคนอยู่ไม่ได้                 |
| numeric    | salary            |    | numeric 12,2                            |
| date       | join_date         |    | วันที่ลวัน ไม่มีเวลา                    |
| boolean    | is_active         |    | true = Active                           |
| date       | last_updated_date |    | ระบบตั้งให้                             |
| int        | version           |    | เพิ่มทีละ 1 ทุกครั้งที่แก้              |
| timestampz | created_at        |    |                                         |
| timestampz | updated_at        |    |                                         |

นอกจากนี้ยังมีตาราง `_prisma_migrations` ที่ Prisma ใช้จดว่า migration ไหนถูก apply แล้ว

ตารางอื่นที่เคยมี ( `users` , `sessions` , `reports` , `integration_state` , `idempotency_keys` , `app_meta` ) ถูกลบไปแล้วด้วย migration สองตัวตอนตัดขอบเขต (D-46 และ D-56) ถ้าเห็นชื่อพวกนี้ในเอกสารเก่าให้ถือว่าเป็นประวัติ

## Prisma คืออะไร

Prisma เป็น **ORM** (Object-Relational Mapping) คือเครื่องมือที่ให้เขียนโค้ด TypeScript แทน SQL ได้ และรู้ชนิดของข้อมูลทุกคอลัมน์ ในโปรเจกต์นี้ Prisma มีสามบทบาท



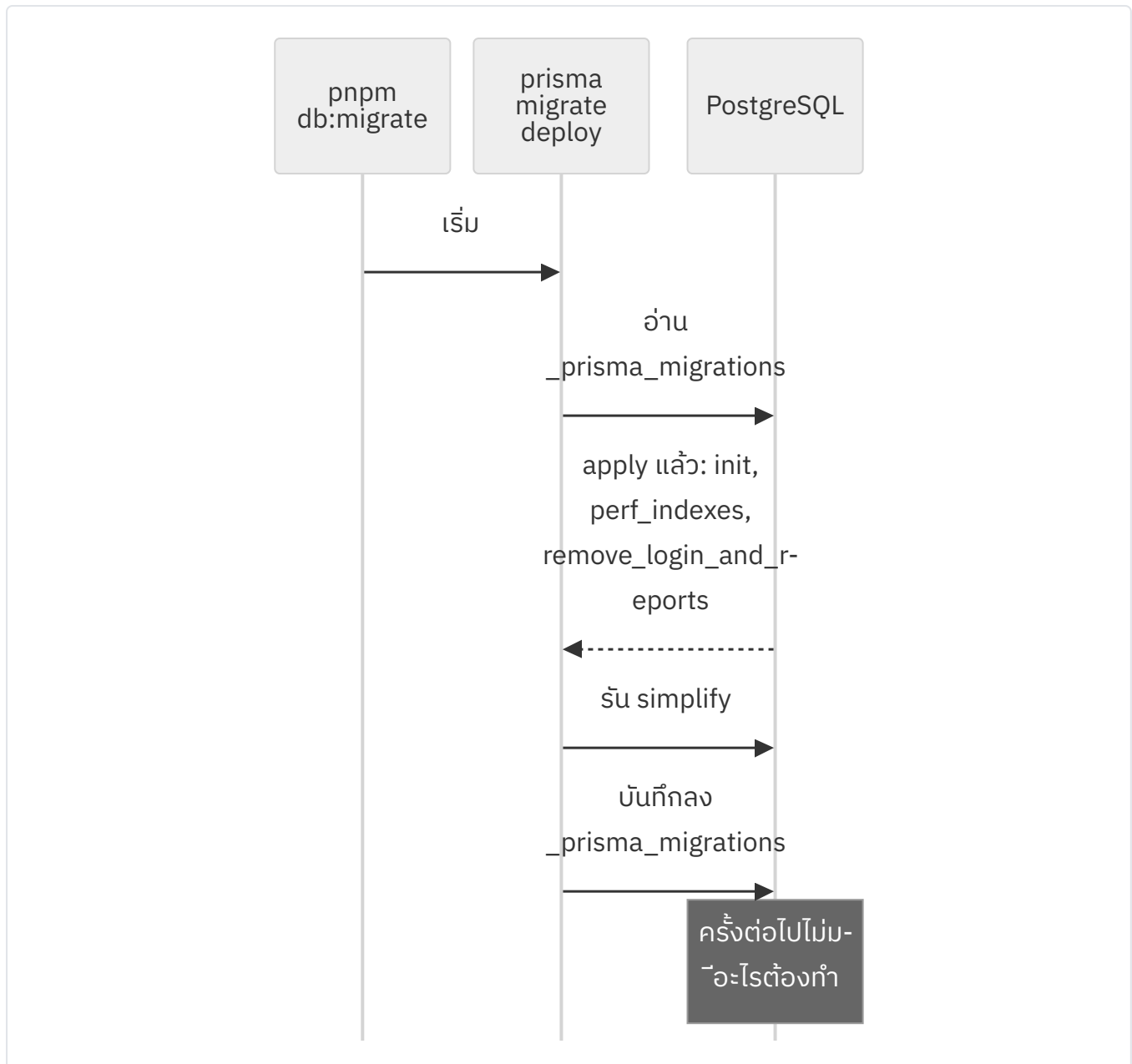
1. **Schema** — `schema.prisma` (`apps/api/prisma/schema.prisma`) บอกว่ามีตารางอะไร คอลัมน์ชนิดไหน ฝั่ง SQL ใช้ชื่อแบบ `snake_case` (เช่น `join_date`) ฝั่งโค้ดใช้ `camelCase` (เช่น `joinDate`) โดยผูกกันด้วย `@map`
2. **Client** — คำสั่ง `prisma generate` อ่าน schema แล้วสร้างโค้ดไว้ใน `apps/api/src/generated/prisma` ซึ่ง `pnpm dev`, `build` และ `typecheck` ทำให้อัตโนมัติ ห้ามแก้ไฟล์ในนั้นด้วยมือ
3. **Migration** — ไฟล์ SQL ที่บอกวิธีเปลี่ยนฐานข้อมูลจากเวอร์ชันหนึ่งไปอีกเวอร์ชัน

`PrismaService` ใน `prisma.service.ts` (`apps/api/src/prisma/prisma.service.ts`) คือ Prisma client ที่ห่อเป็น Nest provider เพื่อให้ inject ได้ (un 2) Prisma 7 ต่อ PostgreSQL ผ่าน driver adapter (`PrismaPg`) ด้วย `DATABASE_URL` ส่วนคำสั่ง CLI อย่าง `migrate deploy` อ่าน URL จาก `prisma.config.ts` (`apps/api/prisma.config.ts`) ซึ่งโหลด `.env` ที่ root เอง

## Migration ทำงานอย่างไร

ลองนึกถึง migration เป็น "บันทึกการต่อเติมบ้าน" แต่ละไฟล์เป็นการต่อเติมหนึ่งครั้ง เรียงตามเวลา ฐานข้อมูลจดไว้ว่าต่อเติมถึงครั้งไหนแล้ว เวลารัน `pnpm db:migrate` ระบบจะทำเฉพาะครั้งที่ยังไม่ได้ทำ

| Migration                                                                                                                  | ทำอะไร                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 20261001000000_init (apps/api/prisma/migrations/20261001000000_init/migration.sql)                                         | สร้างตารางทั้งหมด พร้อม identity column และ CHECK constraint (เช่น <code>salary &gt;= 0</code> , ชื่อต้อง trim แล้ว, <code>join_date</code> อยู่ระหว่างปี 1900–2100) |
| 20261002000000_perf_indexes (apps/api/prisma/migrations/20261002000000_perf_indexes/migration.sql)                         | เพิ่ม index สำหรับกรอง แผนก+สถานะ และ trigram index บน <code>lower(name)</code> (D-32)                                                                               |
| 20261003000000_remove_login_and_reports (apps/api/prisma/migrations/20261003000000_remove_login_and_reports/migration.sql) | ลบตารางของ Login และ AI reports (D-46)                                                                                                                               |
| 20261005000000_simplify (apps/api/prisma/migrations/20261005000000_simplify/migration.sql)                                 | ลบ <code>idempotency_keys</code> และ <code>app_meta</code> (D-56)                                                                                                    |



กติกาสงของโปรเจกต์ (จาก prisma/AGENTS.md (apps/api/prisma/AGENTS.md))

- **append-only** — ห้ามแก้ไฟล์ migration ที่ apply ไปแล้ว ถ้าต้องการเปลี่ยนให้เพิ่มไฟล์ใหม่ ( YYYYMMDDHHMMSS\_<ชื่อ> )
- สิ่งที่ `schema.prisma` เขียนไม่ได้ (identity, CHECK, trigram index) ต้องเขียน SQL เองใน migration แล้วใส่คอมเม้นต์ใน schema ซี่ไป

# Prisma client ใน employees.service.ts

ทุกคำสั่งของ employees ใช้ Prisma client ธรรมดา (D-56 แผนการเขียน SQL เองแบบเดิม) ดู employees.service.ts (apps/api/src/employees/employees.service.ts)

| งาน                                      | คำสั่ง Prisma                                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------------|
| รายการ + ค้นหา + กรอง + เรียง + แบ่งหน้า | <code>\$transaction([count({ where } ), findMany({ where, include, orderBy, skip, take })])</code> |
| ดูคนเดียว                                | <code>findUnique({ where: { id }, include: { department: true } })</code>                          |
| สร้าง                                    | <code>create({ data, include })</code>                                                             |
| แก้ / ลบ                                 | <code>updateMany / deleteMany</code> ที่ <code>where: { id, version }</code> (un 2)                |

`include: { department: true }` ดึงแถวของแผนกมาด้วย API จึงส่ง `departmentName` ได้ ส่วนการเรียงใช้ `map ORDER_BY` ที่แปลงชื่อฟิลด์จาก query เป็น `orderBy` ของ Prisma และเติม `{ id: 'asc' }` เป็นตัวตัดสินเสมอ ลำดับจึงไม่สลับไปมาเมื่อค่าเท่ากัน

## เงินกับวันที่: แปลงทีเดียว

Prisma คำนวณบางชนิดเป็น object ที่ไม่ใช่สิ่งที่เราอยากส่งออก service จึงแปลงทุกแถวผ่าน `toEmployee()` ก่อนส่งกลับ

| คอลัมน์                                             | Prisma ให้มา                                   | API ส่งออก                | วิธีแปลง                                     |
|-----------------------------------------------------|------------------------------------------------|---------------------------|----------------------------------------------|
| <code>salary ( numeric(12,2) )</code>               | object <code>Decimal</code>                    | <code>"72000.00"</code>   | <code>row.salary.toFixed(2)</code>           |
| <code>join_date , last_updated_date ( date )</code> | JavaScript <code>Date</code> เวลาเที่ยงคืน UTC | <code>"2022-11-10"</code> | <code>date.toISOString().slice(0, 10)</code> |

เขาเข้าก็แปลงกลับด้วยฟังก์ชันคู่กัน `fromDateOnly("2026-09-01")` สร้าง `new Date("2026-09-01T00:00:00Z")` และ "วันนี้" สำหรับ Last Updated Date คำนวณด้วย `Intl.DateTimeFormat` เขต `Asia/Bangkok` ไม่ขึ้นกับ timezone ของเครื่อง server

**กับดัก:** อย่าใช้ `toLocaleDateString()` หรือ `getDate()` กับ `Date` ที่ได้จากคอลัมน์ `date` เพราะสองตัวนี้ใช้ timezone ของเครื่อง ในเขตที่อยู่หลัง UTC (เช่นอเมริกา) วันจะถอยไปหนึ่งวัน `toISOString()` อ่านเป็น UTC เสมอจึงได้วันที่ตรง

## ค้นหาชื่อ: ต้อง escape เอง

```
// Prisma passes `contains` to ILIKE as is, so escape % and _ to search for them literally.
name: query.q ? { contains: query.q.replace(/[\%_]/g, '\\$&'), mode: 'insensitive' } :
undefined,
```

`contains` + `mode: 'insensitive'` กลายเป็น `name ILIKE '%คำค้น%'` ใน SQL ซึ่ง `%` แปลว่า "อะไรก็ได้" และ `_` แปลว่า "ตัวอักษรอะไรก็ได้หนึ่งตัว" ถ้าผู้ใช้พิมพ์ `%` แล้วไม่ escape จะได้พนักงานทุกคน บรรทัดนี้จึงเติม `\` หน้า `%`, `_` และ `\` ให้กลายเป็นตัวอักษรธรรมดา มี integration test ยืนยันว่าค้น `%` ได้ 0 แถว (AC-23)

หมายเหตุ: trigram index ใน migration `perf_indexes` สร้างบน `lower(name)` สำหรับ SQL แบบเดิม (`lower(name) LIKE ...`) ส่วน `name ILIKE ...` ที่ Prisma สร้างตอนนี้ไม่ตรงกับ expression ของ index จึงไม่ได้ใช้ (ลอง `EXPLAIN` ดูจะเห็น Seq Scan) กับข้อมูลหลักสิบแถวไม่มีผลอะไร ส่วน index แผนก+สถานะยังถูกใช้อยู่

## แล้วปลอดภัยจาก SQL injection ไหม

ปลอดภัย Prisma client ส่งค่าที่ผู้ใช้พิมพ์เป็น parameter แยกจากตัว SQL ไม่ได้เอาไปต่อ string ส่วนชื่อคอลัมน์ที่ใช้เรียงต้องผ่าน `@IsIn(SORT_FIELDS)` ใน DTO ก่อน แล้วเลือกจาก map `ORDER_BY` เท่านั้น ค่าที่ผู้ใช้ส่งมาไม่มีทางกลายเป็นข้อความ SQL (integration test ลองค้น `' OR 1=1 --` แล้วได้ 0 แถว)

## Seed และ reset

| คำสั่ง                       | ทำอะไร                                                                                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>pnpm db:migrate</code> | apply migration ที่ยังไม่ได้ทำ                                                                                                |
| <code>pnpm db:seed</code>    | ใส่แผนก 4 แผนก (ข้ามที่มีแล้ว) และพนักงาน 5 คนจาก Excel เฉพาะเมื่อตาราง <b>employees</b> ว่าง ถ้ามีพนักงานอยู่แล้วจะไม่แตะเลย |
| <code>pnpm db:reset</code>   | ลบพนักงานทั้งหมด ( <code>TRUNCATE employees RESTART IDENTITY</code> ) แล้ว seed ใหม่ ID ถัดไปกลับเป็น 106                     |

โค้ดอยู่ที่ `seed.ts` (`apps/api/src/seed.ts`) (สคริปต์ที่ `pnpm` เรียกคือ `scripts/seed.ts` (`apps/api/scripts/seed.ts`)) ทุกขั้นตอนอยู่ใน transaction เดียว จุดที่ควรรู้

- อ่านจาก `test-exam-data.json` (`apps/api/prisma/seed-data/test-exam-data.json`) ซึ่งเป็นสำเนาของไฟล์ Excel
- คง **Last Updated Date** เดิมจาก Excel ไว้ ไม่ได้เปลี่ยนเป็นวันนี้ (PRD §4.3)
- Status เกือบตรง ๆ `row.Status === 'Active'` IWS จะ `Boolean("In Active")` ได้ `true`
- หลังใส่ ID 101–105 แล้วตั้ง sequence ด้วย `setval(..., max(id))` คนใหม่จึงได้ 106

`pnpm db:reset` ลบข้อมูลในฐานที่ `DATABASE_URL` ซี่อยู่ ตอน dev คือฐาน dev ของเราเอง ส่วน E2E เรียกคำสั่งเดียวกัน โดยตั้ง `DATABASE_URL` เป็นฐาน test ก่อน และ integration test เรียกฟังก์ชัน `seed(prisma, { reset: true })` ตรง ๆ กับฐาน test

## ลองเอง

ดูข้อมูลพนักงานใน dev โดยตรง (อ่านอย่างเดียว ต่อจากข้างใน container จึงไม่ต้องใส่รหัสผ่าน)

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c 'SELECT id, name, salary, join_date, is_active, version FROM employees ORDER BY id'
```

ดูว่ามีตารางอะไรบ้าง

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c '\dt'
```

ดูว่า migration ไหม apply แล้วบ้าง

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c 'SELECT migration_name, finished_at FROM _prisma_migrations ORDER BY migration_name'
```

---

ต่อไป: un 4 – OpenAPI และ Swagger UI (`docs/learn/04-openapi.md`)

## บท 4 – OpenAPI และ Swagger UI

### OpenAPI คืออะไร

**OpenAPI** คือมาตรฐานสำหรับเขียนเอกสาร API เป็นไฟล์ JSON (หรือ YAML) ที่ทั้งคนและโปรแกรมอ่านได้ในไฟล์บอกว่า

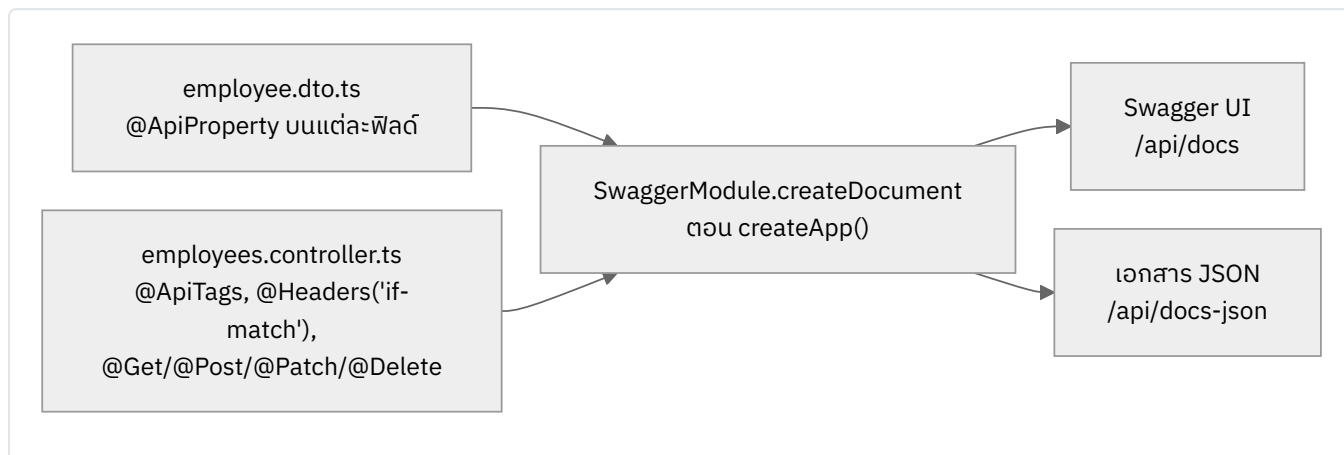
- มี path อะไรบ้าง แต่ละ path รับ method อะไร
- ต้องส่ง parameter, header และ body แบบไหน
- schema ของข้อมูลแต่ละชนิด เช่น `CreateEmployeeDto` มีฟิลด์อะไร ชนิดอะไร บังคับหรือไม่

**Swagger** คือชื่อเดิมของมาตรฐานนี้ และเป็นชื่อชุดเครื่องมือรอบ ๆ มัน เช่น **Swagger UI** หน้าเว็บที่แสดงเอกสารและทดลองยิง API ได้ ส่วน `@nestjs/swagger` คือ library ของ Nest ที่อ่าน decorator ในโค้ดแล้วสร้างเอกสาร OpenAPI ให้เปรียบเทียบง่าย ๆ ได้ว่า OpenAPI คือ **เมนูของร้าน** ที่เขียนละเอียดว่าสั่งอะไรได้ และต้องบอกอะไรบ้าง

### ในโปรเจกต์นี้ เอกสารถูกสร้างตอนแอปเริ่ม

ไม่มีใครเขียนไฟล์ OpenAPI เอง และไม่มีไฟล์ที่ commit ไว้ ทุกครั้งที่ API เริ่ม `createApp()` ใน `app.ts` (`apps/api/src/app.ts`) จะให้ `@nestjs/swagger` อ่าน decorator แล้วสร้างเอกสารเก็บในหน่วยความจำ

```
const openApi = new DocumentBuilder().setTitle('Employee Console API').build();
SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, openApi));
```



เปิดได้สองทาง

- ผ่านเว็บ: <http://localhost:3000/api/docs> (path ขึ้นต้นด้วย `/api` จึงวิ่งผ่าน rewrite ของ Next.js เหมือนคำขออื่น)
- ตรงที่ API: <http://127.0.0.1:3001/api/docs> และไฟล์ JSON ดิบที่ `/api/docs-json` ( `/api/docs-yaml` ก็มี เป็นค่า default ของ `SwaggerModule` )

## ในเอกสารมีอะไร

| Path                | Method             | มาจาก                                                                                                                                                                                                                                              |
|---------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/employees      | GET                | query parameter 7 ตัวจาก <code>ListEmployeesQuery</code> ( <code>q</code> , <code>departmentId</code> , <code>status</code> , <code>page</code> , <code>pageSize</code> , <code>sortBy</code> , <code>sortOrder</code> ) พร้อมค่า default และ enum |
| /api/employees      | POST               | body <code>CreateEmployeeDto</code>                                                                                                                                                                                                                |
| /api/employees/{id} | GET, PATCH, DELETE | PATCH ใช้ body <code>UpdateEmployeeDto</code> และทั้ง PATCH / DELETE มี header <code>If-Match</code> บังคับ                                                                                                                                        |

## ตามรอยฟิลด์ salary หนึ่งฟิลด์

### 1. ใน DTO — employee.dto.ts (apps/api/src/employees/employee.dto.ts)

```
@ApiProperty({ example: '62000.00', description: 'Decimal string, at most 2 decimal places' })
@IsString({ message: 'Salary must be sent as a string, e.g. "65000.00".' })
salary: string;
```

### 2. ใน /api/docs-json

```
"salary": { "type": "string", "example": "62000.00", "description": "Decimal string, at most 2 decimal places" }
```

`@ApiProperty` มีไว้ให้เอกสารอย่างเดียว ส่วนการตรวจสอบจริงเป็นหน้าที่ของ `@IsString` / `@Matches` กับ `ValidationPipe` (บน 2) สองชุดนี้อยู่บนฟิลด์เดียวกัน จึงแก้ไปพร้อมกันได้ง่าย

## สิ่งที่เอกสารไม่ได้บอก

controller ไม่ได้ติดป้ายบอกหน้าตาของ response (เช่น `@ApiOkResponse`) เอกสารจึงบอกแค่ status ที่สำเร็จ (200, 201, 204) ไม่มี schema ของ response และไม่ได้ระบุ error 400/404/409/428 ถ้าอยากรู้หน้าตาจริงให้ดูบน 6 หรือกด "Try it out" ใน Swagger UI กับ GET

## Type ฝั่งเว็บเขียนเอง

เว็บไม่ได้ generate type จาก OpenAPI แต่เขียน interface เองใน api.ts (apps/web/src/lib/api.ts) ให้ตรงกับ `Employee` ใน employees.service.ts (apps/api/src/employees/employees.service.ts)

```
export interface Employee {
  id: number;
  name: string;
  departmentId: string;
  departmentName: string;
  salary: string;
  joinDate: string;
  isActive: boolean;
  lastUpdatedDate: string;
  version: number;
}
```

เหตุผลคือ API มี resource เดียว 5 endpoint การมีแพ็กเกจ generate type แยก พร้อมคำสั่งและการตรวจว่าไฟล์ตรงกัน เป็นการมากกว่าประโยชน์ จึงถูกตัดออกใน D-56

**ก๊าดัก:** เพราะ type เขียนสองที่ ถ้าเปลี่ยนชื่อฟิลด์ใน API แล้วลืมแก้ `lib/api.ts` `pnpm typecheck` จะ **ไม่** ฟ้อง ต้องแก้ทั้งสองฝั่งพร้อมกัน แล้วรัน `pnpm test:e2e` ให้เบราว์เซอร์จริงยืนยัน ของที่ต้องแก้ให้ตรงกันเองแบบนี้มีอีกสองเรื่อง คือ รายการแผนก (`DEPARTMENT_IDS` ใน DTO, `departments.ts` (`apps/web/src/lib/departments.ts`) และไฟล์ seed) กับกฎของฟอร์ม (zod schema ใน `employee-form.tsx` (`apps/web/src/components/employees/employee-form.tsx`) เขียนตามกฎใน DTO แต่ API ยังเป็นผู้ตัดสินใจสุดท้าย)

## ลองเอง

เปิด <http://localhost:3000/api/docs> ในเบราว์เซอร์ กด `GET /api/employees` → **Try it out** → ใส่ `q` เป็น `john` → **Execute**

ดูเอกสาร JSON ดิบ

```
curl -s http://127.0.0.1:3001/api/docs-json
```

**ก๊าดัก:** ในหน้า `PATCH` และ `DELETE` ต้องกรอกช่อง header `if-match` เป็น version ปัจจุบัน เช่น `"1"` (ไม่ใส่ → 428, ไม่ตรง → 409) — "Try it out" แก่ข้อมูลจริงในฐาน dev ถ้าเผลอแก้ ค้นได้ด้วย `pnpm db:reset`

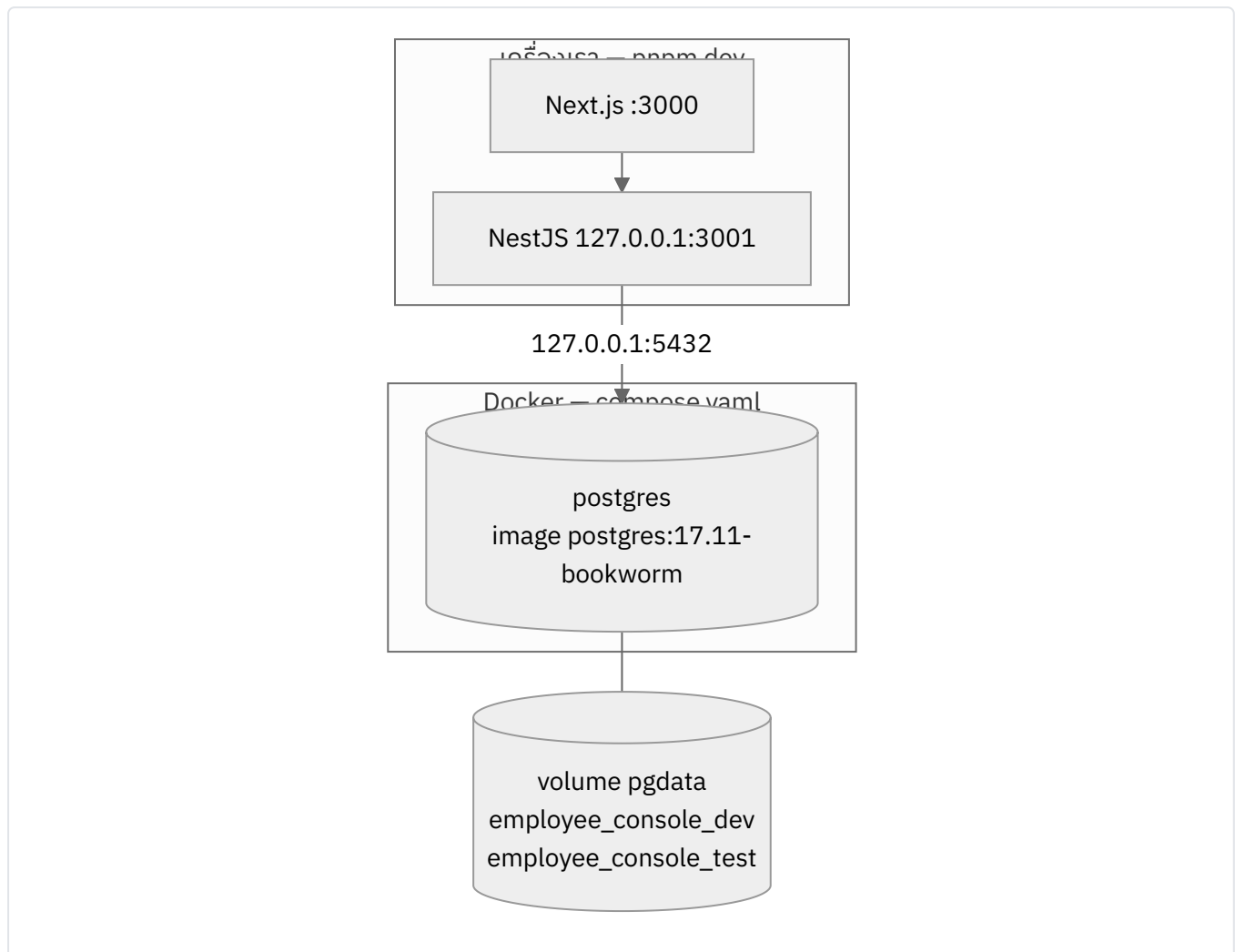
ต่อไป: un 5 — Docker (<docs/learn/05-docker.md>)

## un 5 – Docker

ในโปรเจกต์นี้ Docker มีหน้าที่เดียว คือรัน PostgreSQL ให้เหมือนกันทุกเครื่อง โดยไม่ต้องติดตั้ง PostgreSQL ลงเครื่องเอง ส่วนเว็บและ API รันบนเครื่องโดยตรงด้วย `pnpm dev` เพราะ hot reload เร็วกว่า และโปรเจกต์ไม่ได้สร้าง image ของตัวเอง (ตัดออกใน D-56)

### คำศัพท์ 4 คำ

| คำ               | เปรียบเทียบ                                                   | ในโปรเจกต์นี้                                                                  |
|------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Image</b>     | แม่พิมพ์ หรือกล่องข้าวแช่แข็งที่ปิดฝาแล้ว เปลี่ยนข้างในไม่ได้ | <code>postgres:17.11-bookworm</code> image ทางการของ PostgreSQL ที่ดาวน์โหลดมา |
| <b>Container</b> | ขนมที่อบออกมาจากแม่พิมพ์ = image ที่กำลังรันอยู่              | <code>employee-console-postgres-1</code>                                       |
| <b>Volume</b>    | ตู้เก็บของที่อยู่นอก container ทั้ง container แล้วของยังอยู่  | <code>pgdata</code> เก็บไฟล์ข้อมูลของ PostgreSQL                               |
| <b>Compose</b>   | ใบสั่งงานที่บอกว่าจะเปิด container อะไร ตั้งค่าอย่างไร        | <code>compose.yaml</code>                                                      |



## อ่าน `compose.yaml` ที่ละบรรทัด

```
name: employee-console                                # ชื่อ project → container ชื่อ employee-console-
postgres-1

services:
  postgres:
    image: postgres:17.11-bookworm                    # pin เวอร์ชันชัดเจน ไม่ใช่ latest
    restart: unless-stopped                           # Docker เปิดให้ใหม่เอง ยกเว้นเราสั่งหยุด
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres} # อ่านจาก .env ถ้าไม่มีใช้ postgres
      POSTGRES_DB: employee_console_dev                # สร้างฐาน dev ให้ตอนเปิดครั้งแรก
      TZ: UTC
    ports:
      - "127.0.0.1:${POSTGRES_HOST_PORT:-5432}:5432"  # เปิดเฉพาะเครื่องเรา ไม่เปิดให้ LAN
    volumes:
      - pgdata:/var/lib/postgresql/data                # ข้อมูลอยู่ใน volume ไม่หายเมื่อลบ container
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"] # ถามว่ารับการเชื่อมต่อได้หรือยัง
      interval: 5s
      timeout: 3s
      retries: 20

volumes:
  pgdata:
```

- **ports** — รูปแบบคือ ที่อยู่บนเครื่องเรา:พอร์ตบนเครื่องเรา:พอร์ตใน container การใส่ `127.0.0.1` นำหน้าทำให้เครื่องอื่นในเครือข่ายต่อเข้ามาไม่ได้
- **healthcheck** — Docker รัน `pg_isready` ทุก 5 วินาที จนกว่าจะตอบว่าพร้อม สถานะจึงเปลี่ยนจาก `starting` เป็น `healthy`
- `${...}` — Compose อ่านตัวแปรจากไฟล์ `.env` ที่ root ให้เอง

## คำสั่ง `pnpm` ที่เรียก Docker

ดูใน `package.json` ที่ root

| คำสั่ง                   | ทำอะไร                                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| <code>pnpm dev:up</code> | <code>docker compose up -d --wait</code> → <code>pnpm db:migrate</code> → <code>pnpm db:seed</code> |
| <code>pnpm down</code>   | <code>docker compose down</code> ลบ container แต่ <b>ไม่ลบ volume</b> ข้อมูลจึงยังอยู่              |

`-d` คือรันเบื้องหลัง ส่วน `--wait` คือรอจน container เป็น `healthy` ก่อนค่อยจบคำสั่ง ขึ้น migrate ที่ตามมาจึงไม่ไปต่อฐานข้อมูลที่ยังไม่พร้อม

**ก๊บนิด:** `docker compose down -v` (มี `-v`) จะลบ volume ด้วย ข้อมูลทั้งฐาน dev และฐาน test หายหมด ถ้าแค่อยากได้ข้อมูล 5 records เดิมคืน ใช้ `pnpm db:reset` พอ

**ก๊าดัก:** `POSTGRES_PASSWORD` ถูกใช้แค่ตอนสร้าง volume ครั้งแรก เปลี่ยนใน `.env` ทีหลังจะไม่มีผลกับ volume เดิม  
และถ้าเครื่องมี PostgreSQL ตัวอื่นใช้พอร์ต 5432 อยู่ ให้ตั้ง `POSTGRES_HOST_PORT` แล้วแก้พอร์ตใน  
`DATABASE_URL` และ `TEST_DATABASE_URL` ให้ตรงกัน (`configuration.md` ([docs/configuration.md](#)))

## ลองเอง

ดูสถานะ container (ควรเห็น `healthy`)

```
docker compose ps
```

ดู log ของ PostgreSQL 20 บรรทัดล่าสุด

```
docker compose logs --tail 20 postgres
```

ดู volume ที่เก็บข้อมูล (ชื่อจริงคือชื่อ project นำหน้า)

```
docker volume ls --filter name=employee-console_pgdata
```

---

ต่อไป: บท 6 – แนวคิดสำคัญของโปรเจกต์ ([docs/learn/06-key-concepts.md](#))

## บท 6 – แนวคิดสำคัญของโปรเจกต์

---

บทนี้รวมเรื่องที่ทำให้โปรเจกต์นี้ต่างจาก CRUD ทั่วไป และเป็นเรื่องที่มีมักถูกถามตอนนำเสนอ

### 1. ก้นการเขียนทับกัน: `version` + `If-Match`

---

#### ปัญหา: lost update

สองคนเปิดหน้า Edit ของ Bob พร้อมกัน คนแรกแก้เงินเดือน คนที่สองแก้แผนก ถ้าระบบไม่ตรวจสอบอะไรเลย คนที่กด Save ทีหลังจะเขียนทับสิ่งที่คนแรกแก้ไขโดยไม่มีใครรู้ตัว

#### วิธีแก้: optimistic concurrency

ทุกแถวมีคอลัมน์ `version` เริ่มที่ 1 และเพิ่มขึ้นทีละ 1 ทุกครั้งที่แก้ไขสำเร็จ

1. ตอนอ่าน API ส่ง `version` มาใน JSON ของพนักงาน เช่น `"version": 1`
2. ตอนแก้ไขหรือลบ client ต้องส่ง version ที่ตัวเองเห็นกลับมาใน header `If-Match: "1"`
3. ถ้าในฐานข้อมูลยังเป็น 1 → แก้ได้ แล้ว version กลายเป็น 2
4. ถ้าในฐานข้อมูลกลายเป็น 2 ไปแล้ว → ตอบ **409** ให้ผู้ใช้โหลดข้อมูลล่าสุดก่อน

เรียกว่า "optimistic" เพราะมองโลกในแง่ดีว่าส่วนใหญ่ไม่ชนกัน จึงไม่ล๊อคอะไรไว้ตลอดเวลาที่เปิดฟอร์ม แค่ตรวจตอนบันทึก



`If-Match` รับทั้ง `"3"` และ `3` (D-23) ส่วน header `ETag` ที่เห็นใน response เป็นของ Express สร้างอัตโนมัติ ไม่ใช่ version (un 2)

ฝั่งเว็บทำสามอย่างให้ระบบนี้ใช้งานได้จริง

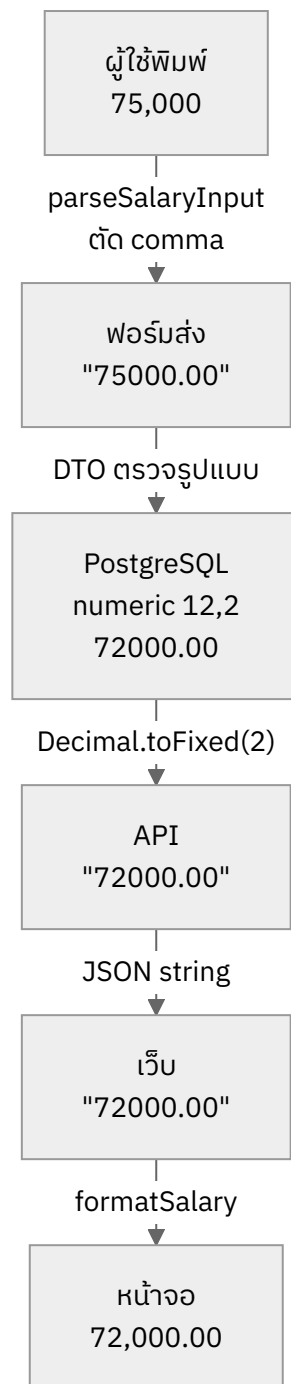
- หน้า Edit ส่ง `version` ของข้อมูลที่แสดงอยู่ในฟอร์มไปใน `If-Match` ผ่าน `useUpdateEmployee` (`queries.ts` (`apps/web/src/lib/queries.ts`))
- `QueryClient` ปิด `refetchOnWindowFocus` (`providers.tsx` (`apps/web/src/app/providers.tsx`)) ไม่อย่างนั้นสลับแท็บกลับมาแล้วฟอร์มจะถูกแทนด้วยข้อมูลใหม่ (รวม version ใหม่) ทั้งที่ผู้ใช้กำลังพิมพ์อยู่ และการตรวจจะไม่มีความหมาย
- ได้ 409 → แสดงแถบเตือนพร้อมปุ่ม **Reload latest** กดแล้ว `refetch` และฟอร์ม reset เป็นค่าล่าสุด (`edit-employee-view.tsx` (`apps/web/src/app/(console)/employees/[id]/edit/edit-employee-view.tsx`)) ส่วนกล่องยืนยันการลบ (`delete-employee-dialog.tsx` (`apps/web/src/components/employees/delete-employee-dialog.tsx`)) แสดง toast error เมื่อได้ 409 และถ้าได้ 404 จะบอกว่า "This employee was already deleted."

ดูโค้ด: `parseIfMatch` ใน `employees.controller.ts` (`apps/api/src/employees/employees.controller.ts`), `update / remove` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`), integration test "needs If-Match (428) and refuses a stale version (409)" ใน `employees.test.ts` (`apps/api/test/integration/employees.test.ts`) และ E2E "two tabs editing the same record" ใน `employee-crud.spec.ts` (`tests/e2e/specs/employee-crud.spec.ts`) (AC-16)

## 2. เงินเดือนเป็น string ทุกชั้น

```
0.1 + 0.2 // 0.30000000000000004
```

`number` ของ JavaScript เป็น floating point เก็บทศนิยมบางค่าได้ไม่ตรง เมื่อเป็นเรื่องเงินจึงไม่ใช่ `number` เลย



- ฐานข้อมูลใช้ `numeric(12,2)` เก็บได้ถึง 9,999,999,999.99 แบบแม่นยำ
- API รับเป็น string ที่ตรงกับ `^\d{1,10}(\.\d{1,2})?$` เท่านั้น ส่ง `75000` (ตัวเลข JSON) จะได้ 400 "Salary must be sent as a string" ส่ง `"65000"` ได้ และฐานข้อมูลเก็บเป็น `65000.00`
- การแสดง `#,##0.00` ทำในเว็บด้วยการจัดการ string ล้วน (`format.ts` (`apps/web/src/lib/format.ts`), `salary.ts` (`apps/web/src/lib/salary.ts`))
- ช่วง Salary format ตอน blur ไม่ใช่ตอน focus เพราะเคยเกิดบั๊ก `62000.0063000.00` ที่ Playwright จับได้ (D-31)

### 3. วันที่แบบไม่มีเวลา ( YYYY-MM-DD )

Join Date และ Last Updated Date เป็น "วันที่ล้วน" ไม่มีเวลาและไม่มี timezone

```
new Date('2023-01-15') // = 2023-01-15T00:00:00Z (เที่ยงคืน UTC)
new Date('2023-01-15').toLocaleDateString('en-US', { timeZone: 'America/New_York' })
// "1/14/2023" ← วันเลื่อนถอยไป 1 วัน
```

โปรเจกต์จึงส่งเป็น string "2023-01-15" ตลอดทาง (D-05)

- **API** แปลงคอลัมน์ `date` ที่ Prisma ให้มาเป็น `Date` ด้วย `toISOString().slice(0, 10)` ซึ่งอ่านเป็น UTC เสมอ (un 3)
- **เว็บ** แสดงผลด้วย `formatDateOnly()` ที่ใช้ regex แยกปี เดือน วัน ไม่สร้าง `Date` เลย (`format.ts` (`apps/web/src/lib/format.ts`)) และช่อง Join Date เป็น `<input type="date">` ที่ให้ค่าเป็น `YYYY-MM-DD` อยู่แล้ว
- **E2E test** เปิดเบราว์เซอร์ในหลาย timezone แล้วตรวจว่าวันที่ไม่เลื่อน (AC-11 ใน `dates-and-layout.spec.ts` (`tests/e2e/specs/dates-and-layout.spec.ts`))

**Last Updated Date** ระบบตั้งให้เป็น "วันนี้" ตามปฏิทินของ `Asia/Bangkok` ด้วย `todayInBangkok()` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`) ไม่ว่าเครื่อง server จะตั้ง timezone อะไร เช่น ตอน 00:30 ที่กรุงเทพ (ยังเป็นวันก่อนหน้าใน UTC) ระบบจะใช้วันที่ของกรุงเทพ ซึ่งมี integration test ตรวจกรณีนี้ ส่วน timestamp อย่าง `created_at` เก็บเป็น UTC

### 4. รูปแบบ response และ error

response ที่สำเร็จคือข้อมูลตรง ๆ ไม่มีการห่อ

```
{ "id": 104, "name": "Bob Brown", "departmentId": "engineering", "departmentName":
"Engineering",
  "salary": "72000.00", "joinDate": "2022-11-10", "isActive": false, "lastUpdatedDate": "2026-
03-01", "version": 1 }
```

หน้า list มีข้อมูลการแบ่งหน้าอยู่ข้างรายการ

```
{ "items": [ { "id": 101, "...": "..." } ], "page": 1, "pageSize": 20, "total": 5,
"totalPages": 1 }
```

error ใช้รูปแบบมาตรฐานของ NestJS

```
{ "message": ["Salary must be sent as a string, e.g. \"65000.00\"."], "error": "Bad Request",
"statusCode": 400 }
```

| Status | message                                                                            | เกิดเมื่อ                                                                             |
|--------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 400    | array ของข้อความ                                                                   | body หรือ query ผิด, ส่งพาดที่ไม่รู้จัก ( <code>property id should not exist</code> ) |
| 400    | <code>Validation failed (numeric string is expected)</code>                        | id ใน path ไม่ใช่ตัวเลข                                                               |
| 404    | <code>This employee does not exist or was deleted.</code>                          | ไม่มี ID นี้                                                                          |
| 409    | <code>This employee was changed by another user. Reload the latest version.</code> | version ไม่ตรง                                                                        |
| 428    | <code>Send If-Match with the employee version you edited.</code>                   | ไม่ส่ง <code>If-Match</code> (body นี้ไม่มี key <code>error</code> )                  |

ฝั่งเว็บ `api()` ใน `api.ts` (`apps/web/src/lib/api.ts`) แปลงทุก error เป็น `ApiError(status, message)` ถ้า `message` เป็น array ก็ต่อเป็นประโยคเดียว แล้ว `errorMessage()` เลือกข้อความที่จะแสดง: status ต่ำกว่า 500 แสดงข้อความของ error นั้นเอง (ข้อความจาก server หรือ "Could not reach the server..." ที่ `api()` สร้างให้เมื่อเน็ตหลุดซึ่งใช้ status 0) ส่วน 5xx แสดงข้อความกลาง ๆ "The server could not complete the request. Try again."

## 5. กด Save โดยไม่ได้ทำอะไร = ไม่ส่งคำขอ

หน้า Edit ใช้ `changedFields()` เทียบค่าในฟอร์มกับข้อมูลเดิม แล้วส่งเฉพาะฟิลด์ที่เปลี่ยน ถ้าไม่มีอะไรเปลี่ยนแปลงจะขึ้น toast "No changes to save." และไม่เรียก API (AC-14) ตัว API เองไม่ได้ตรวจเรื่องนี้ ทุก `PATCH` ที่ผ่านการตรวจจะเพิ่ม version และตั้ง Last Updated Date เป็นวันนี้เสมอ

## 6. ข้อมูลตั้งต้นจาก Excel ต้องไม่ผิด

| จุด                                                   | ทำอะไร                                                                                                                                         |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Bob Brown มีสถานะ <code>In Active</code> (มีช่องว่าง) | seed เทียบ <code>row.Status === 'Active'</code> จึงได้ <code>false</code> ห้ามใช้ <code>Boolean("In Active")</code> เพราะได้ <code>true</code> |
| Last Updated Date เดิมของ 5 records                   | seed เก็บค่าจาก Excel ไม่เปลี่ยนเป็นวันนี้                                                                                                     |
| ID 101-105                                            | seed ใส่ ID เดิม แล้วตั้ง sequence ให้คนใหม่เริ่มที่ 106                                                                                       |
| รูปแบบ <code>65,000.00</code>                         | เป็นการแสดงผลเท่านั้น ค่าจริงคือ <code>"65000.00"</code>                                                                                       |

ดู `seed.ts` (`apps/api/src/seed.ts`) และ unit test `seed.test.ts` (`apps/api/test/unit/seed.test.ts`)

## 7. สถานะของหน้า list อยู่ใน URL

ค้นหา กรอง เรียง และเลขหน้าเก็บใน query string เช่น `/employees? q=john&departmentId=engineering&status=inactive&page=2` จึง reload, กด back/forward หรือส่งลิงก์ให้คนอื่น แล้วเห็นผลเดียวกัน (PRD §8.3)

`list-params.ts` (`apps/web/src/lib/list-params.ts`) มีสองฟังก์ชันหลัก

- `readListParams()` อ่าน URL ค่าที่ผิด (เช่น `pageSize=999`) จะกลับเป็นค่า default
- `toSearch()` สร้าง `?...` เฉพาะค่าที่ต่างจาก default ใช้ทั้งกับ URL ของหน้าและกับคำขอ `GET /api/employees` เพราะ default ของเว็บกับของ API ตั้งไว้เท่ากัน หน้าแรกจึงเป็นแค่ `/employees`

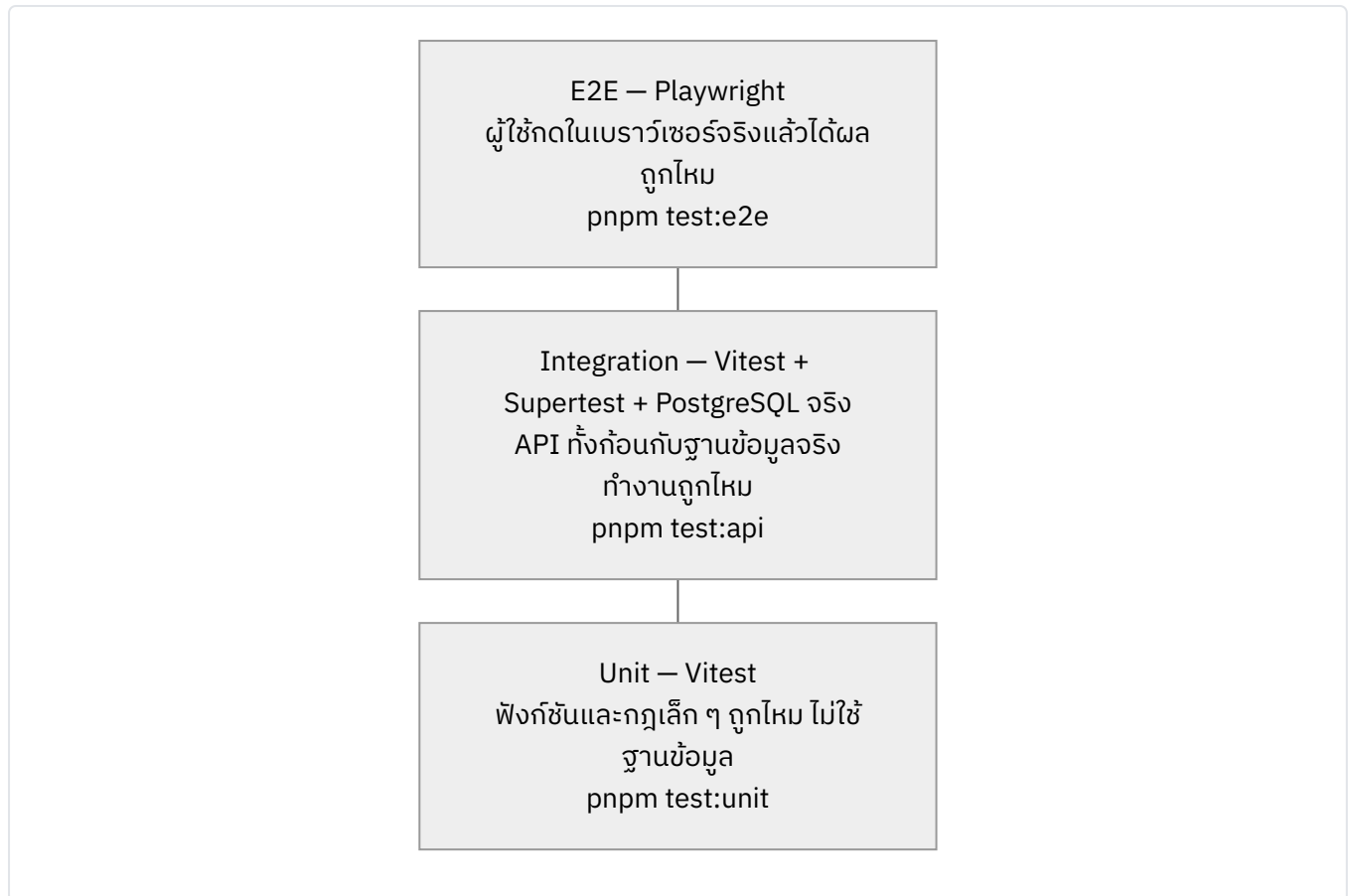
เปลี่ยนตัวกรองหรือการเรียงเมื่อไรจะกลับไปหน้า 1 เสมอ (`employees-view.tsx`  
(`apps/web/src/app/(console)/employees/employees-view.tsx`))

---

ต่อไป: บท 7 — การทดสอบ (`docs/learn/07-testing.md`)

## บท 7 – การทดสอบ

### ภาพรวม: พีระมิดของ test



ยิ่งอยู่ล่าง ยิ่งเร็วและมีจำนวนเยอะ ยิ่งอยู่บน ยิ่งช้าแต่ใกล้ผู้ใช้จริง แต่ละชั้นจับปัญหาคนละแบบ ไม่ได้แทนกัน

| ชั้น        | เครื่องมือ         | อยู่ที่                                                | ต้องมี PostgreSQL | ตัวอย่างสิ่งที่ตรวจ                                                                                             |
|-------------|--------------------|--------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------|
| Unit        | Vitest             | apps/api/test/unit,<br>apps/web/src/lib/format.test.ts | ไม่               | DTO ปฏิเสธ salary "65000.999", seed map In Active เป็น false, formatSalary("65000.00") ได้ "65,000.00"          |
| Integration | Vitest + Supertest | apps/api/test/integration                              | ใช่               | สร้างได้ ID 106 และวันนี้ตามเวลากรุงเทพ, ส่ง id มาใน body ได้ 400, แก่พร้อมกัน 3 คำขอผ่านแค่ 1, คำน % ได้ 0 แถว |
| E2E         | Playwright         | tests/e2e/specs                                        | ใช่               | สร้าง → แก่ → ลบ, สองแท็บชนกันได้แถบเตือน, วันที่ไม่เลื่อนในหลาย timezone, จอ 375 px ไม่ล้น                     |

## Unit test — เร็วที่สุด

---

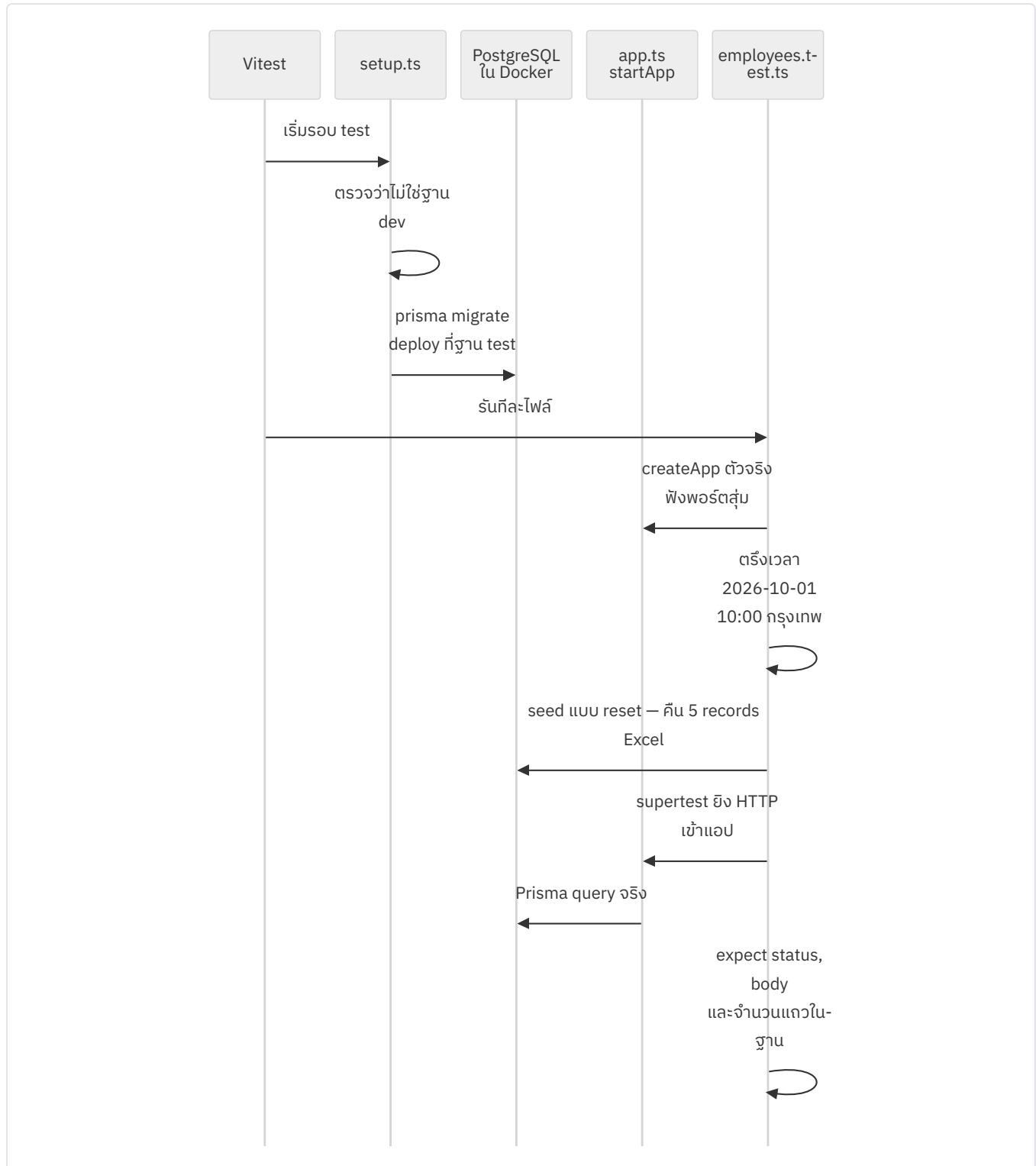
ทดสอบโค้ดที่ไม่ต้องใช้ฐานข้อมูลหรือ HTTP เช่น `employee-dto.test.ts` (`apps/api/test/unit/employee-dto.test.ts`) เรียก `validate()` ของ `class-validator` กับ DTO ตรง ๆ จึงลองค่าผิดได้หลายสิบแบบในเวลาไม่กี่วินาที และ `seed.test.ts` (`apps/api/test/unit/seed.test.ts`) ตรวจสอบการแปลงแถว Excel

คำสั่งนี้รัน unit test ของทั้ง API และเว็บ

```
pnpm test:unit
```

## Integration test – API ทิ้งก้อนกับ PostgreSQL จริง

ไม่ใช่ฐานข้อมูลจำลอง เพราะพฤติกรรมสำคัญหลายอย่าง ( `UPDATE ... WHERE version` ที่ชนกัน, `numeric`, `date`, `ILIKE` ) จำลองให้เหมือนจริงไม่ได้



- `setup.ts` (`apps/api/test/integration/setup.ts`) เป็น global setup รันครั้งเดียวก่อนทุกไฟล์ ปฏิเสธถ้า `TEST_DATABASE_URL` ี่ฐานเดียวกับ `DATABASE_URL` แล้ว migrate ฐาน test
- `app.ts` (`apps/api/test/integration/app.ts`) เปลี่ยน `DATABASE_URL` เป็นฐาน test แล้วเปิดแอปด้วย `createApp()` ตัวเดียวกับของจริง

- `employees.test.ts` (`apps/api/test/integration/employees.test.ts`) ใช้ `vi.useFakeTimers({ toFake: ['Date'] })` ตรึง "ตอนนี้" ไว้ ผลเรื่อง Last Updated Date จึงเหมือนเดิมทุกวัน (ปลอมแค่ `Date` ส่วน timer อื่นยังเดินจริง) และ `seed(prisma, { reset: true })` ก่อนทุก test ให้ ID ถัดไปเป็น 106 เสมอ
- ใช้ Vitest + SWC (`unplugin-swc` ใน `vitest.config.ts` (`apps/api/vitest.config.ts`)) ไม่ใช้ esbuild เพราะ Nest DI ต้องใช้ decorator metadata (D-13)

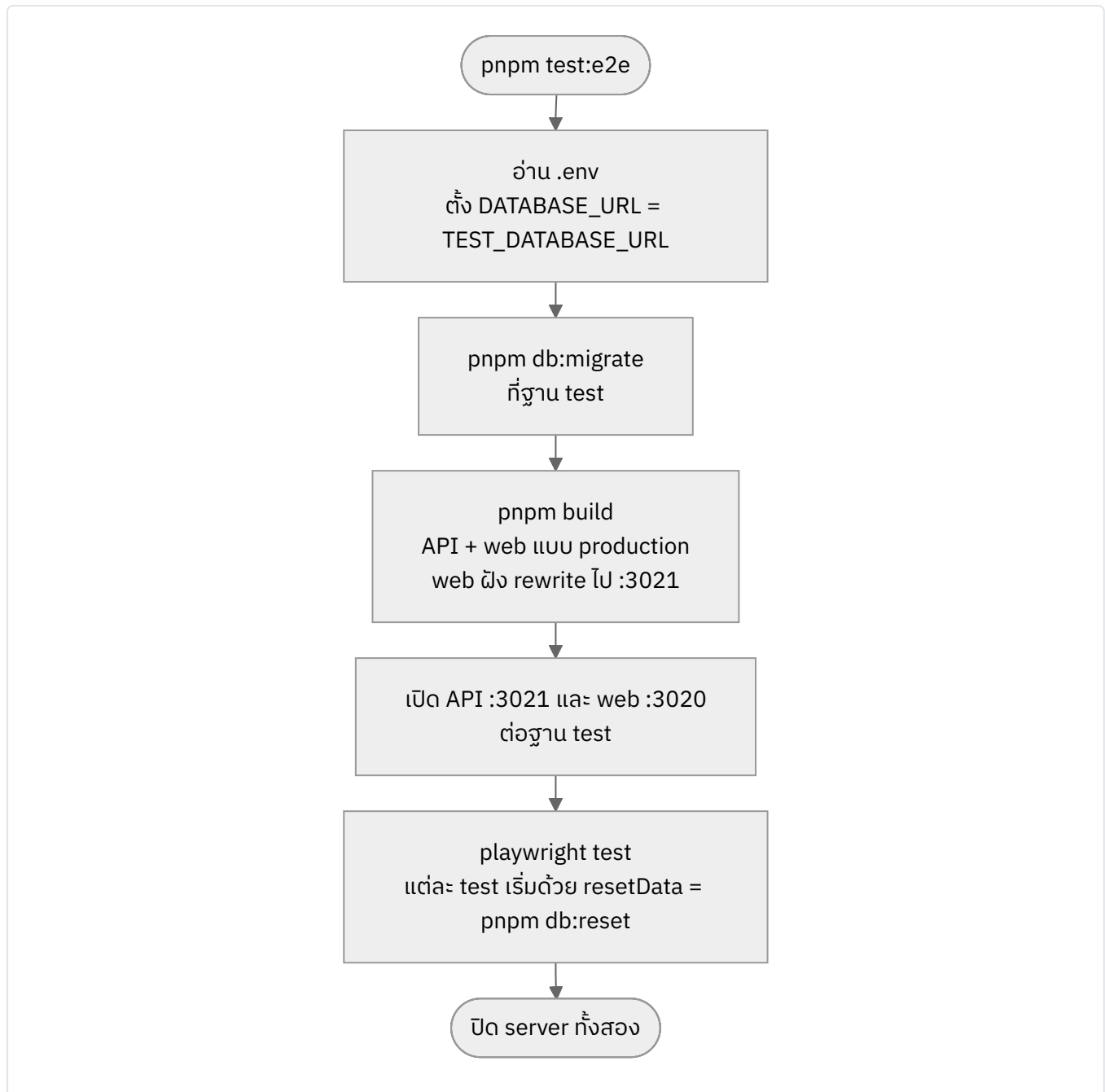
ต้องเปิด PostgreSQL ไว้ก่อน

```
pnpm dev:up
```

```
pnpm test:api
```

## E2E test — Playwright

**Playwright** เปิดเบราว์เซอร์จริง (Chromium) แล้วคลิก พิมพ์ และตรวจหน้าจอเหมือนผู้ใช้ runner คือ `tests/e2e/run.mjs`



ตัวอย่าง test ใน `employee-crud.spec.ts` (`tests/e2e/specs/employee-crud.spec.ts`)

- seed data: 5 records with all 7 fields; Bob Brown is In Active (AC-01, AC-03, AC-26)
- create → edit → delete journey (AC-04, AC-12, AC-13, AC-17)
- saving without changes is a no-op (AC-14)
- two tabs editing the same record: the stale tab gets a conflict (AC-16)

และใน `dates-and-layout.spec.ts` (`tests/e2e/specs/dates-and-layout.spec.ts`)

- dates do not shift in `<timezone>` (AC-11)
- mobile 375px: no page overflow, stacked filters, usable form (AC-35)

- keyboard only: focus order, visible focus, checkbox, dialog Escape, submit with Enter (AC-35)

```
pnpm test:e2e
```

รันบาง test ได้ด้วยการส่ง argument ต่อให้ Playwright

```
pnpm test:e2e --grep "seed data"
```

เก็บภาพหน้าจอที่ 375 / 1024 / 1440 px ไว้ด้วย

```
E2E_SCREENSHOT_DIR=./screenshots pnpm test:e2e
```

## รหัส AC-xx คืออะไร

AC ย่อมาจาก **Acceptance Criteria** คือข้อกำหนดที่ต้องผ่านใน prd.md (docs/prd.md) §16 ชื่อ test อ้างรหัสนี้ไว้ ทำให้ตอบได้ว่า "ข้อกำหนดข้อนี้ถูกทดสอบที่ไหน" ด้วยการค้นหา เช่น

```
grep -rn "AC-16" apps/api/test tests/e2e/specs
```

## ก่อนบอกร่างงานเสร็จ

```
pnpm lint && pnpm typecheck && pnpm test:unit && pnpm build
```

- แก้ API → รัน `pnpm test:api` ด้วย
- แก้ UI หรือ flow → รัน `pnpm test:e2e` ด้วย
- แก้ชนิดข้อมูลของ API → แก้ `api.ts` (apps/web/src/lib/api.ts) ของเว็บให้ตรง (typecheck ไม่จับให้ — un 4)

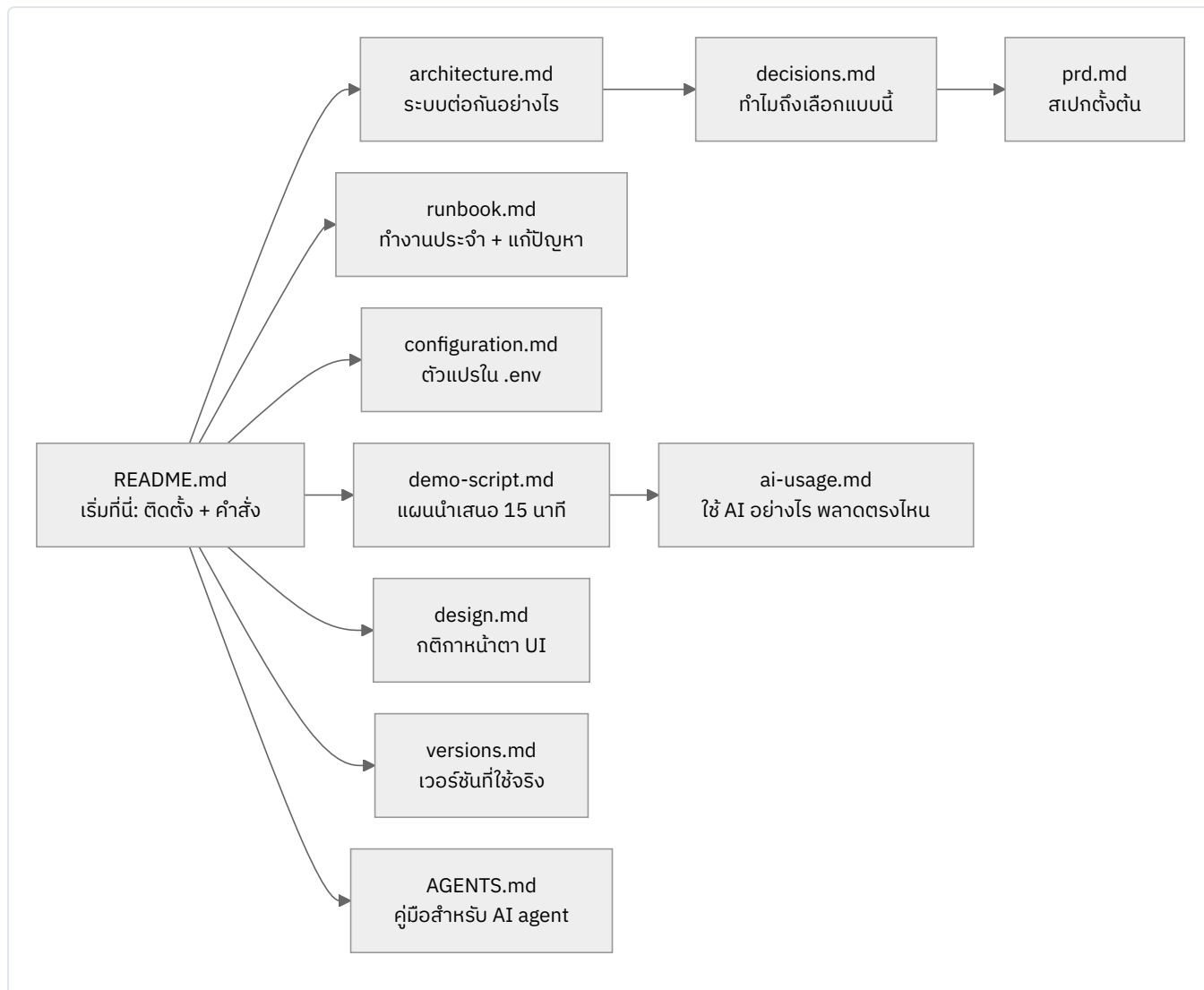
**ก๊าดัก:** อย่ารัน `playwright test` ตรง ๆ ให้ใช้ `pnpm test:e2e` เสมอ เพราะ runner เป็นคน build แอป ตั้ง `DATABASE_URL` เป็นฐาน test และเปิด server บนพอร์ต 3020/3021 ถ้ารันตรง ๆ จะไม่มี server ให้ทดสอบ และ `resetData()` ใน `fixtures.ts` (tests/e2e/specs/fixtures.ts) จะหยุดทันทีเพราะไม่เห็น `TEST_DATABASE_URL` ที่ runner ส่งให้

**ก๊าดัก:** `pnpm test:api` กับ `pnpm test:e2e` ใช้ฐาน `employee_console_test` ฐานเดียวกันและ reset ข้อมูลก่อนทุก test อย่ารันสองคำสั่งนี้พร้อมกัน ไม่อย่างนั้นจะลบข้อมูลของกันและกันจน test ล้มแบบสุ่ม

ต่อไป: un 8 – แผนที่เอกสาร (docs/learn/08-docs-guide.md)

## บท 8 – แผนที่เอกสาร ฉบับอ่านง่าย

เอกสารในโปรเจกต์เขียนให้ "แม่นยำ" มากกว่า "อ่านง่าย" บทนี้สรุปแต่ละไฟล์เป็นภาษาธรรมดา บอกว่าไฟล์นั้นมีไว้ทำอะไร ควรเปิดตอนไหน และใจความสำคัญคืออะไร



### README.md – ประตูหน้าบ้าน

README.md บอกว่าโปรเจกต์ทำอะไร ต้องติดตั้งอะไร และมีคำสั่งอะไรบ้าง

- **อ่านเมื่อ:** วันแรกที่เปิดโปรเจกต์ หรือลืมว่าคำสั่งไหนทำอะไร
- **ใจความ:** ต้องมี Node 24, pnpm และ Docker Desktop ขั้นตอนเริ่มคือ `pnpm install` → `cp .env.example .env` → `pnpm dev:up` → `pnpm dev`

## AGENTS.md และ CLAUDE.md — คู่มือสำหรับ AI

AGENTS.md ที่ root และในแต่ละ folder หลัก (เช่น apps/api (apps/api/AGENTS.md), apps/web (apps/web/AGENTS.md)) เขียนให้ AI coding assistant อ่านก่อนแก้ไขโค้ด ส่วน CLAUDE.md แค่ import ไฟล์ AGENTS.md

- **อ่านเมื่อ:** อยากรู้ "กฎที่อ่านจากโค้ดไม่ออก" เช่น salary ต้องเป็น string, import ต้องลงท้าย .js, ห้ามแก้ไข migrationเก่า
- **ใจความ:** คนก็อ่านได้ดีเหมือนกัน เพราะเป็นรายการกับดักที่สั้นที่สุดของแต่ละส่วน

## architecture.md — ระบบต่อกันอย่างไร

architecture.md (docs/architecture.md) มีแผนภาพ service, ตารางไฟล์ของ API, endpoint ทั้งหมด, ลำดับชั้นของ request และ data model

- **อ่านเมื่อ:** ต้องอธิบายระบบให้คนอื่นฟัง หรือจะเพิ่ม feature ใหม่
- **ใจความ:** เบราร์เชอร์คุยกับ Next.js ที่เดียว Next.js ส่ง /api/\* ต่อให้ NestJS ซึ่งเป็น Controller → Service → Prisma และสรุปจุดที่ตั้งใจให้ถูกต้อง (เงิน, วันที่, การแก้กับกัน, การค้นหา) ซึ่งตรงกับบท 2, 3 และ 6 ของคู่มือนี้

## configuration.md — ตัวแปรใน .env

configuration.md (docs/configuration.md) อธิบายตัวแปรทั้งหมด

- **อ่านเมื่อ:** ต่อฐานข้อมูลไม่ได้, พอร์ตชน หรือจะเพิ่มตัวแปรใหม่
- **ใจความ:** มีไฟล์เดียวคือ .env ที่คัดลอกจาก .env.example ตัวหลักคือ DATABASE\_URL (ฐาน dev) กับ TEST\_DATABASE\_URL (ฐาน test ที่ถูกลบทุกครั้งที่รัน test) ตัวเสริมเช่น PORT และ API\_INTERNAL\_URL ซึ่งถูกฝังตอน next build เพิ่มตัวแปรใหม่ต้องแก้ไฟล์นี้และ .env.example ด้วย

## runbook.md — คู่มือปฏิบัติงาน

runbook.md (docs/runbook.md) คือ "ทำอะไรเมื่อ..."

- **อ่านเมื่อ:** ตั้งเครื่องครั้งแรก, อยากรันข้อมูลเป็น 5 records ก่อน demo, รัน test หรือเจอปัญหา
- **ใจความ:** ตั้งเครื่อง → pnpm dev → pnpm db:reset → คำสั่ง test และตารางปัญหาที่พบบ่อย เช่น พอร์ต 3001 ถูกใช้, ต่อฐานข้อมูลไม่ได้, รหัสผ่านไม่ตรงกับ volume เดิม

## decisions.md — ทำไมถึงเลือกแบบนี้

decisions.md (docs/decisions.md) เป็นบันทึกการตัดสินใจ (decision log) แต่ละข้อมีรหัส `D-xx` พร้อมเหตุผล D-01 ถึง D-12 มาจาก PRD ส่วน D-13 ขึ้นไปตัดสินใจระหว่างลงมือทำ ทักษะคือ **เพิ่มแถวใหม่เท่านั้น ห้ามแก้แถวเก่า** ถ้าเปลี่ยนใจให้เขียนแถวใหม่ว่าแทนที่ข้อไหน

- **อ่านเมื่อ:** มีคนถามว่า "ทำไมไม่ใช้ X" หรือเห็นโค้ดแปลก ๆ แล้วสงสัย
- **อ่านข้อไหนก่อน: D-56** เพราะเป็นข้อที่ทำให้โปรเจกต์หน้าตาเป็นแบบปัจจุบัน และบอกว่าแทนที่ข้อเก่าข้อไหนบ้าง

จัดกลุ่มให้อ่านง่าย

| กลุ่ม                         | ข้อ                                                                                                                                                                    | สรุปสั้น ๆ                                                                                                                                                         |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ขอบเขต                        | D-08, D-12, <b>D-46</b> , D-52, <b>D-56</b>                                                                                                                            | UI อังกฤษ/เอกสารไทย; ไม่ทำ Redis, K8s; ตัด Login และ AI reports (D-46), ชุด performance test (D-52), แล้ว ตัด CI/CD, staging และของกันฟังก์ที่จอกย์ไม่ได้ขอ (D-56) |
| Stack และ เวอร์ชัน            | D-01, D-13–D-16, D-47                                                                                                                                                  | Next 16 + Nest 12 + Postgres 17 + Prisma 7; pin เวอร์ชันที่เข้ากันได้จริง; test ใช้ Vitest + SWC                                                                   |
| ความถูกต้องของข้อมูล          | D-04, D-05, D-31, D-32                                                                                                                                                 | เงินเป็น decimal string, วันที่เป็น string, ช่อง Salary format ตอน blur, index แผนก+สถานะ                                                                          |
| การเขียนพร้อมกันและการลบ      | D-06, D-07 (ส่วน <code>version</code> + <code>If-Match</code> ), D-23                                                                                                  | ลบจริงหลังยืนยัน, version + If-Match, รับ <code>"3"</code> และ <code>3</code>                                                                                      |
| เอกสาร                        | D-48, D-49                                                                                                                                                             | ลดเนื้อหา PRD ที่ตัดออก และคืนคู่มือ <code>docs/learn/</code>                                                                                                      |
| ประวัติ (ถูกแทนที่หรือยกเลิก) | D-03, D-09, D-11, D-18, D-24–D-28, D-38–D-40 (ยกเลิกโดย D-46/D-52) และข้อที่ D-56 ระบุดูว่าแทนที่ เช่น D-10, D-17, D-19, D-20, D-21, D-29, D-30, D-33, D-34, D-41–D-44 | Google OAuth, n8n + Gemini, performance test, Jenkins, staging, rate limit, <code>@Rule()</code> , SQL ตรง ๆ, Idempotency-Key ฯลฯ                                  |

## design.md — ทักษะหน้าตา UI

design.md (docs/design.md) อธิบายว่าทำไม UI หน้าตาเหมือนโปรแกรม ERP/HR ขององค์กร (ตารางแนบ มีเส้นกริดครบ) แทนที่จะเป็น dashboard สำเร็จรูป

- **อ่านเมื่อ:** จะแก้หน้าตาเว็บ
- **ใจความ**
  - สืออยู่ใน `apps/web/src/app/globals.css` เป็นโทนเขียว Chememan สีสแดงใช้กับการลบและ error เท่านั้น
  - ฟอนต์ IBM Plex Sans Thai ระบุเดียว ตัวเลขใช้ `tabular-nums`
  - ไม่ใช่ของที่ทำให้ดูเป็น template: sidebar ไอคอน, การ์ดลอยมุมโค้ง, badge มีกรอบ, หัวข้อตัวใหญ่, ตัวเลขสรุปตัวโต
  - Accessibility: ทุก input มี label, สถานะไม่พึ่งสีอย่างเดียว, ใช้คีย์บอร์ดได้ครบ

## prd.md — สเปกตั้งต้น

prd.md (docs/prd.md) ยาวราว 1,300 บรรทัด เขียนก่อนเริ่มโค้ดเพื่อสั่งงาน AI โค้ดอ้างอิงเป็น PRD §... และรหัส REQ-xx, USR-xx, AC-xx หัวไฟล์มีหมายเหตุบอกว่าหัวข้อไหนถูกตัดโดย D-46, D-52 และ D-56

- **อ่านเมื่อ:** อยากรู้ว่าข้อกำหนดเดิมคืออะไร หรือเห็น PRD §8.3 ในคอมเมนต์แล้วอยากตามไปอ่าน
- **ไม่ต้องอ่านทั้งเล่ม** ใช้ตารางนี้เลือก

| ส่วน                       | สถานะ                                   | อ่านเพื่อ                                                                                                                            |
|----------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| §3–§4 โจทย์และข้อมูล Excel | ใช้อยู่                                 | รู้ว่าโจทย์คืออะไร และกับดักของข้อมูล (Bob Brown In Active, วันที่เดิมต้องคงไว้)                                                     |
| §7 Architecture            | ใช้อยู่                                 | ทำไมแยก Next/Nest                                                                                                                    |
| §8 หน้าจอและ user journey  | ใช้อยู่                                 | หน้า list, create/edit/delete ต้องทำงานอย่างไร                                                                                       |
| §9 Data model และ กฎธุรกิจ | ใช้อยู่                                 | validation, วันเวลา, concurrency, seed                                                                                               |
| §10 API contract           | บางส่วน                                 | ใช้ได้เรื่อง endpoint, search/filter/sort และ If-Match; envelope, error code, Idempotency-Key และ endpoint departments ถูกตัด (D-56) |
| §11 Authentication         | เหลือแค่หัวข้อ (D-46)                   | —                                                                                                                                    |
| §12 AI report, n8n, Gemini | เหลือแค่หัวข้อ (D-46)                   | —                                                                                                                                    |
| §13 Environment และคำสั่ง  | บางส่วน                                 | setup / doctor และ health/readiness ถูกตัด (D-56)                                                                                    |
| §14 CI/CD                  | ประวัติ (D-56)                          | —                                                                                                                                    |
| §15 Performance            | เหลือแค่หัวข้อ (D-52)                   | —                                                                                                                                    |
| §16 Acceptance criteria    | ใช้อยู่ ยกเว้นส่วนที่ผูกกับของที่ตัดออก | รหัส AC-xx ที่ test อ้างถึง                                                                                                          |

## versions.md — เวอร์ชันที่ใช้จริง

versions.md (docs/versions.md) บันทึกเวอร์ชันที่ติดตั้งและรันจริง ไม่ใช่ที่คาดหวัง

- **อ่านเมื่อ:** มีคนถามว่าใช้เวอร์ชันอะไร หรือทำไมไม่อัปเดต
- **ใจความ:** เหตุผลที่ไม่ใช้ latest ทุกตัว เช่น TypeScript 5.9.3 (ไม่ใช่ 7 เพราะ @nestjs/swagger ยังไม่รองรับ), Prisma 7.10.0 (latest เป็น 8.0 RC), ESLint 9 (plugin ของ Next ยังไม่รองรับ 10)

## ai-usage.md — ใช้ AI อย่างไร และ AI พลาดตรงไหน

ai-usage.md (docs/ai-usage.md) บันทึกวิธีสั่งงาน AI และปัญหาจริงที่เจอระหว่างพัฒนา พร้อมสาเหตุและวิธีแก้

- **อ่านเมื่อ:** เตรียมตอบคำถามเรื่องการใช้ AI ตอนสัมภาษณ์
- **ใจความ:** ส่ง PRD ให้ AI → ตรวจสอบความเข้ากันของเวอร์ชันก่อน → ทำทีละชิ้นพร้อม test → ให้ test เป็นตัวตัดสิน ไม่ใช่อ่านโค้ด บางแถวในตารางปัญหาเป็นเรื่องของส่วนที่ตัดออกไปแล้ว ไฟล์ระบุไว้ว่าแถวไหนเป็นประวัติ

## demo-script.md — แผนนำเสนอ 15 นาที

demo-script.md (docs/demo-script.md) คือบทนำเสนอ

- **อ่านเมื่อ:** ก่อนวันนำเสนอ
- **ใจความ:** สิ่งที่ต้องเตรียม, ลำดับนำเสนอ (หน้า Employees → CRUD ตามสคริปต์ → เปิดโค้ด → test และการใช้ AI → Q&A), ผลที่ต้องเห็นในแต่ละขั้น, ทางแก้เมื่อมีปัญหาหน้างาน และโจทย์ข้อ Live Coding ซึ่งบท 2 อธิบายลำดับการแก้ไข คำถามที่ควรตอบได้อยู่ในบท 9 พร้อมคำตอบ

---

ต่อไป: บท 9 — เตรียมตอบคำถาม (docs/learn/09-qa-prep.md)

## บท 9 – เตรียมตอบคำถาม

คำถามชุดแรกมาจากรายการ "คำถามที่ควรตอบได้" ใน `demo-script.md` (`docs/demo-script.md`) ชุดที่สองเป็นคำถามที่คนไม่คุ้นกับ NestJS หรือ OpenAPI มักถูกถาม ทุกคำตอบมีไฟล์ให้เปิดประกอบ

วิธีใช้: อ่านคำตอบแล้วลองพูดด้วยคำของตัวเอง ไม่ต้องท่องจำ ถ้าโดนถามสักให้เปิดไฟล์ที่ระบุไว้

### คำถามจาก demo-script

#### ทำไมแยก Next.js กับ NestJS

แยกเพื่อให้กฎของข้อมูลมีที่อยู่ทีเดียว Next.js ทำแค่ UI กับส่งต่อ `/api/*` ไม่ต่อฐานข้อมูลและไม่มี Server Actions ทำ CRUD ซ้ำ การตรวจสอบข้อมูล, กฎ version และการแปลงเงิน/วันที่อยู่ใน NestJS ทั้งหมด ผลคือทดสอบ API ได้ตรง ๆ ด้วย integration test โดยไม่ต้องผ่านหน้าเว็บ มี Swagger UI อธิบาย API ให้ และเบราวเซอร์เห็นแค่ origin เดียวจึงไม่ต้องเปิด CORS

เปิดประกอบ: `next.config.ts` (`apps/web/next.config.ts`), `architecture.md` (`docs/architecture.md`)

#### ทำไมเงินเป็น decimal string

`number` ของ JavaScript เป็น floating point บวกทศนิยมแล้วเพี้ยนได้ เช่น  $0.1 + 0.2$  ไม่เท่ากับ  $0.3$  พอดี ฐานข้อมูลจึงเก็บเป็น `numeric(12,2)` Prisma คีนมาเป็น `Decimal` แล้ว service แปลงเป็น string ด้วย `toFixed(2)` ขาเข้า DTO รับเฉพาะ string ที่ตรงรูปแบบ `^\d{1,10}(\.\d{1,2})?$` ถ้าส่งเป็นตัวเลขมาจะได้ 400 การใส่ comma `#,##0.00` ทำตอนแสดงผลในเว็บด้วยการจัดการ string อย่างเดียว ค่าเงินจึงไม่เคยผ่าน float เลยตั้งแต่ฐานข้อมูลจนถึงฟอร์ม

เปิดประกอบ: `toEmployee()` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`), `salary` ใน `employee.dto.ts` (`apps/api/src/employees/employee.dto.ts`), `salary.ts` (`apps/web/src/lib/salary.ts`), D-04

#### ป้องกัน lost update อย่างไร

ใช้ optimistic concurrency ทุกแถวมี `version` ที่ส่งไปกับข้อมูลพนักงาน ตอนแก้ไขหรือลบ client ต้องส่ง `version` ที่ตัวเองเห็นมาใน header `If-Match` ไม่ส่งได้ 428 ผัง service ใช้ `updateMany` ที่มีเงื่อนไข `where: { id, version }` ซึ่งเป็นคำสั่ง UPDATE เดียวที่ตรวจสอบและเขียนพร้อมกัน ถ้าไม่โดนสักแถวจึงเช็กต่อว่าไม่มี ID นี้ (404) หรือ `version` เปลี่ยนไปแล้ว (409) มี integration test ยิงแก้พร้อมกัน 3 คำขอด้วย `version` เดียวกัน ผ่านแค่คำขอเดียว และ E2E test เปิดสองแท็บแก้ record เดียวกัน แท็บที่ช้ากว่าได้แถบเตือนพร้อมปุ่ม Reload latest (AC-16)

เปิดประกอบ: `update()` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`), un 6 หัวข้อ 1 (`docs/learn/06-key-concepts.md`)

## ทำไมตัดหลายอย่างออก (Login, AI report, CI/CD ฯลฯ)

โจทย์หลักคือ List, Search/Filter และ CRUD ของข้อมูลจาก Excel ที่รันในเครื่องได้ และโค้ดต้องอธิบายและแก้สดได้ ของหลายอย่างเคยทำไว้ตาม PRD ตั้งต้นแต่ตัดออกเป็นสามรอบ บันทึกลงไว้ทุกครั้ง

- **D-46** ตัดระบบ Login (Google OAuth, Admin/Viewer) และรายงาน AI (n8n + Gemini)
- **D-52** ตัดชุด performance test
- **D-56** ตัด delivery (Jenkins, staging, Docker image, Postman) และของกันพังระดับ production ที่โจทย์ไม่ได้ขอ (Idempotency-Key, rate limit, envelope ของ response, health check, generated client) แล้วเขียน API ใหม่ด้วย Prisma client และ DTO มาตรฐานของ NestJS

สิ่งที่ยังคงไว้คือเรื่องที่ทำให้ข้อมูลถูกต้อง ได้แก่ `version` + `If-Match`, เงินเป็น string, วันที่เป็น `YYYY-MM-DD` และ "วันนี้" ตามเวลากรุงเทพ

เปิดประกอบ: D-46, D-52, D-56 ใน `decisions.md` (`docs/decisions.md`), migration

`remove_login_and_reports`

(`apps/api/prisma/migrations/20261003000000_remove_login_and_reports/migration.sql`) และ `simplify`

(`apps/api/prisma/migrations/20261005000000_simplify/migration.sql`)

## ทำไม seed ไม่ stamp วันที่ใหม่

Last Updated Date ใน Excel เป็นข้อมูลตั้งต้นของโจทย์ ถ้า seed เปลี่ยนเป็นวันที่รัน ข้อมูลจะไม่ตรงต้นฉบับ PRD §4.3 ระบุให้คง ID และวันที่แก้ไขเดิมของทั้ง 5 รายการ seed จึงใส่ค่าจาก Excel ตรง ๆ และทำงานเฉพาะเมื่อตาราง employees ว่าง จึงไม่ทับข้อมูลที่ถูกแก้ไขแล้ว วันที่จะเปลี่ยนเป็น "วันนี้ตามเวลากรุงเทพ" ก็ต่อเมื่อผู้ใช้สร้างหรือแก้ไข record ผ่านระบบเท่านั้น ถ้าอยากเริ่มใหม่ใช้ `pnpm db:reset`

เปิดประกอบ: `seed.ts` (`apps/api/src/seed.ts`), `seed.test.ts` (`apps/api/test/unit/seed.test.ts`), integration test "seeding again adds nothing and keeps edited values"

## AI ทำอะไรผิดจริงบ้าง

บันทึกไว้ใน `ai-usage.md` พร้อมหลักฐาน ตัวอย่างที่ยังเกี่ยวกับโค้ดปัจจุบัน:

1. จะติดตั้ง Prisma 8 RC เพราะ `latest` ชี้ไป RC ต้อง pin 7.10.0
2. รันโค้ด Nest ด้วย `tsx` แล้ว DI พัง เพราะ esbuild ไม่สร้าง decorator metadata จึงใช้ SWC ใน test
3. ช่อง Salary format ตอน focus ทำให้พิมพ์ใหม่แล้วข้อความต่อท้าย `62000.0063000.00` Playwright จับได้
4. ตอนเปลี่ยนเป็น Prisma `contains` ค้นหา `%` แล้วได้ทุกแถว เพราะ Prisma ไม่ escape ให้ integration test AC-23 จับได้
5. `PATCH { "name": null }` ได้ 500 แทน 400 เพราะ `PartialType` ข้ามค่า `null` แก้ด้วย `skipNullProperties: false`

บทเรียนคือให้ test เป็นตัวตัดสิน ไม่เชื่อแค่การอ่านโค้ดหรือคำอธิบายของ AI

เปิดประกอบ: `ai-usage.md` (`docs/ai-usage.md`)

## คำถามพื้นฐานที่อาจโดนถาม

### NestJS ต่างจาก Express อย่างไร

Nest ใช้ Express อยู่ข้างใน แต่เพิ่มโครงสร้าง: Module รวมของที่เกี่ยวข้อง, Controller รับ HTTP, Service ทำงานจริง และมี Dependency Injection สร้างของให้ งานที่ต้องทำกับทุก request เช่นตรวจสอบข้อมูล ทำผ่าน Pipe (`ValidationPipe` + DTO) และ error ก็โยนเป็น exception แล้ว Nest แปลงเป็น response ให้ โค้ดจึงไม่กองรวมในที่เดียว

เปิดประกอบ: un 2 ([docs/learn/02-nestjs.md](#)), `app.ts` ([apps/api/src/app.ts](#))

### Dependency Injection มีประโยชน์อะไรในโปรเจกต์นี้

`EmployeesService` ไม่ได้สร้างการเชื่อมต่อฐานข้อมูลเอง แต่ขอ `PrismaService` ผ่าน constructor Nest สร้างให้ตัวเดียวแล้วใช้ร่วมกันทั้งแอป และเรียก `onModuleDestroy()` ปิดการเชื่อมต่อให้ตอนแอปหยุด ถ้าวันหนึ่งอยากเปลี่ยนตัวที่ใช้คุยกับฐานข้อมูลใน test ก็ทำผ่าน DI ได้โดยไม่แก้ service

เปิดประกอบ: `prisma.service.ts` ([apps/api/src/prisma/prisma.service.ts](#)), `prisma.module.ts` ([apps/api/src/prisma/prisma.module.ts](#))

### OpenAPI / Swagger ใช้ทำอะไร

`@nestjs/swagger` อ่าน decorator ใน controller และ DTO แล้วสร้างเอกสาร OpenAPI ตอนแอปเริ่ม เปิดดูและลองยิงได้ที่ `/api/docs` เอกสารจึงตรงกับโค้ดเสมอเพราะมาจากที่เดียวกัน ส่วน type ผังเว็บเขียนเองใน `lib/api.ts` เพราะ API มีแค่ 5 endpoint การ generate client แยกเป็นการมากกว่าประโยชน์ (ตัดออกใน D-56) แลกกับการที่ต้องจำแก้ type สองฝั่งพร้อมกัน

เปิดประกอบ: un 4 ([docs/learn/04-openapi.md](#)), `api.ts` ([apps/web/src/lib/api.ts](#))

### ป้องกัน SQL injection อย่างไร

ใช้ Prisma client ซึ่งส่งค่าที่ผู้ใช้พิมพ์เป็น parameter แยกจาก SQL เสมอ ชื่อคอลัมน์ที่ใช้เรียงต้องผ่าน `@IsIn(SORT_FIELDS)` แล้วเลือกจาก map `ORDER_BY` เท่านั้น คำค้นหาชื่อ escape `%` กับ `_` ก่อนส่งให้ `contains` และ `ValidationPipe` ปฏิกิริยา query ที่ไม่รู้จัก มี integration test ค้น `' OR 1=1 --` และ `%` แล้วได้ 0 แถว (AC-23)

เปิดประกอบ: `list()` ใน `employees.service.ts` ([apps/api/src/employees/employees.service.ts](#)), un 3 ([docs/learn/03-database-prisma.md](#))

## ถ้ากดสร้างแล้วเน็ตหลุด จะได้ข้อมูลซ้ำไหม

อาจได้ ถ้าคำขอไปถึง server และบันทึกแล้วแต่คำตอบหายระหว่างทาง แล้วผู้ใช้กด Save ซ้ำ เดิมเคยมี Idempotency-Key กันกรณีนี้ แต่ตัดออกใน D-56 เพราะโจทก์ไม่ได้ขอและเป็นแอปในเครื่อง หน้าเว็บจะขึ้นข้อความว่าติดต่อ server ไม่ได้ ผู้ใช้ควรเปิดรายการดูก่อนกดซ้ำ ถ้าซ้ำจริงก็ลบได้ปกติ ถ้าระบบต้องใช้จริงหลายคน ควรนำ Idempotency-Key กลับมา

เปิดประกอบ: D-56 ใน decisions.md (docs/decisions.md), new-employee-view.tsx (apps/web/src/app/(console)/employees/new/new-employee-view.tsx)

## test ไปลบข้อมูล dev ได้ไหม

ปกติไม่ได้ test ใช้ฐานแยก TEST\_DATABASE\_URL integration test ตั้ง DATABASE\_URL เป็นฐาน test ก่อนเปิดแอป และ setup.ts ปฏิเสธถ้าสอง URL เป็นฐานเดียวกัน ส่วน E2E runner ก็ตั้ง DATABASE\_URL เป็นฐาน test ให้ทั้ง API และคำสั่ง reset แต่การป้องกันมีแค่นี้ ถ้าตั้ง TEST\_DATABASE\_URL ซ้ำไปฐานอื่นที่มีข้อมูลจริง test จะลบข้อมูลนั้น จึงต้องตั้งค่าตาม .env.example

เปิดประกอบ: setup.ts (apps/api/test/integration/setup.ts), run.mjs (tests/e2e/run.mjs), configuration.md (docs/configuration.md)

## มี index อะไรบ้าง และใช้หลักฐานอะไรเลือก

มีสองตัวใน migration perf\_indexes ตัวแรกเป็น B-tree uu (department\_id, is\_active) ใช้กับตัวกรองแผนก+สถานะ EXPLAIN แสดงว่าใช้ index นี้จริง ตัวที่สองเป็น trigram index uu lower(name) ซึ่งทำไว้ตอน query ยังเป็น SQL ที่เขียนเองแบบ lower(name) LIKE ... หลังเปลี่ยนเป็น Prisma contains ที่สร้าง name ILIKE ... index นี้ไม่ถูกใช้แล้ว ตอบตรง ๆ ได้ว่ากับข้อมูลหลักสับแถวไม่มีผล ถ้าข้อมูลโตจริงควรวัดด้วย EXPLAIN ANALYZE แล้วสร้าง trigram index uu name ใน migration ใหม่ (migration เก่าห้ามแก้)

เปิดประกอบ: perf\_indexes migration (apps/api/prisma/migrations/20261002000000\_perf\_indexes/migration.sql), D-32, un 3 (docs/learn/03-database-prisma.md)

## ถ้าต้องเพิ่ม feature ใหม่ จะทำตรงไหน

ฝั่ง API สร้าง folder apps/api/src/<feature> ที่มี module, controller, service และ dto แบบเดียวกับ employees แล้วเพิ่ม module เข้า imports ของ AppModule ถ้าต้องมีตารางใหม่ก็แก้ schema.prisma และเพิ่ม migration ใหม่ ฝั่งเว็บเพิ่ม type ใน lib/api.ts, hook ใน lib/queries.ts แล้วทำ component ใน components/<feature> และหน้าใน app/ ปิดท้ายด้วย integration test และ E2E ถ้าเป็นแค่ตัวกรองใหม่ของ employees ดูสูตรใน un 2

เปิดประกอบ: app.module.ts (apps/api/src/app.module.ts), queries.ts (apps/web/src/lib/queries.ts), un 2 หัวข้อสูตร (docs/learn/02-nestjs.md)

## ถ้าโดนถามสิ่งที่ไม่รู้

---

- บอกตรง ๆ ว่าส่วนนั้นยังไม่ได้ลงลึก แล้วชี้ว่าจะหาคำตอบจากที่ไหน เช่น "การตัดสินใจนี้บันทึกไว้ใน decisions.md ขอเปิดดูครับ"
- เปิดโค้ดจริงแทนการเดา ทุกเรื่องในคู่มือนี้มีลิงก์ไปไฟล์
- ใช้ test เป็นหลักฐาน: ค้นหาคำ `AC-xx` แล้วเปิด test ที่เกี่ยวข้องให้ดู

กลับไป: สารบัญ ([docs/learn/README.md](#))

# อภิธานศัพท์

เรียงตามหมวด แต่ละคำบอกความหมายสั้น ๆ และบทที่อธิบายละเอียด

## เว็บและ HTTP

| คำ                 | ความหมาย                                                                                                               | un                                |
|--------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Request / Response | คำขอที่ส่งไป และคำตอบที่ได้กลับ ประกอบด้วย method, path, header, body / status code, header, body                      | 1 (docs/learn/01-big-picture.md)  |
| Method             | คำกริยาของ HTTP: <code>GET</code> อ่าน, <code>POST</code> สร้าง, <code>PATCH</code> แก้บางส่วน, <code>DELETE</code> ลบ | 1 (docs/learn/01-big-picture.md)  |
| Status code        | ตัวเลขสามหลักบอกผล เช่น 200 สำเร็จ, 404 ไม่พบ, 409 ชนกัน, 428 ขาด <code>If-Match</code>                                | 1 (docs/learn/01-big-picture.md)  |
| Header             | ข้อมูลประกอบ request/response เช่น <code>Content-Type</code> , <code>If-Match</code>                                   | 1 (docs/learn/01-big-picture.md)  |
| JSON               | รูปแบบข้อมูลที่ส่งไปมา หน้าตาเหมือน object ของ JavaScript                                                              | 1 (docs/learn/01-big-picture.md)  |
| REST               | แนวคิดที่มองข้อมูลเป็น resource มี URL ของตัวเอง แล้วใช้ method บอกการกระทำ                                            | 1 (docs/learn/01-big-picture.md)  |
| Origin             | scheme + host + port เช่น <code>http://localhost:3000</code> เบราว์เซอร์แยกสิทธิ์ตาม origin                            | 1 (docs/learn/01-big-picture.md)  |
| CORS               | กลไกที่อนุญาตให้หน้าเว็บเรียก origin อื่น โปรเจกต์นี้ไม่เปิดเพราะใช้ origin เดียว                                      | 1 (docs/learn/01-big-picture.md)  |
| Rewrite / Proxy    | Next.js รับคำขอ <code>/api/*</code> แล้วส่งต่อให้ NestJS โดยเบราว์เซอร์ไม่รู้                                          | 1 (docs/learn/01-big-picture.md)  |
| If-Match           | header ที่ client ส่ง version ที่ตัวเองเห็นกลับมา เพื่อให้ server ตรวจสอบแก้หรือลบ                                     | 6 (docs/learn/06-key-concepts.md) |
| ETag               | header บอก "รุ่น" ของ response ในโปรเจกต์นี้ Express สร้างให้อัตโนมัติจากเนื้อหา <b>ไม่ใช่</b> เลข version             | 2 (docs/learn/02-nestjs.md)       |
| Query string       | ส่วนหลัง <code>?</code> ของ URL เช่น <code>?q=john&amp;page=2</code> หน้า list เก็บสถานะไว้ตรงนี้                      | 6 (docs/learn/06-key-concepts.md) |

## NestJS

| คำ                        | ความหมาย                                                                                                                                    | un                          |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| Decorator                 | ป้าย <code>@...</code> ที่แปะบน class/method/property บอก framework ว่าโค้ดนั้นคืออะไร                                                      | 2 (docs/learn/02-nestjs.md) |
| Module                    | กล่องที่รวม controller และ provider ของเรื่องเดียวกัน                                                                                       | 2 (docs/learn/02-nestjs.md) |
| Controller                | class ที่รับ HTTP request แกะข้อมูล แล้วเรียก service                                                                                       | 2 (docs/learn/02-nestjs.md) |
| Provider / Service        | class ที่ Nest สร้างแล้วแจกจ่ายผ่าน DI; service คือที่อยู่ของงานจริง                                                                        | 2 (docs/learn/02-nestjs.md) |
| Dependency Injection (DI) | ขของที่ต้องใช้ผ่าน constructor แล้ว Nest สร้างและส่งให้                                                                                     | 2 (docs/learn/02-nestjs.md) |
| <code>@Global()</code>    | module ที่ทุก module ใช้ของได้โดยไม่ต้อง import เช่น <code>PrismaModule</code>                                                              | 2 (docs/learn/02-nestjs.md) |
| Global prefix             | คำนำหน้าทุก route ตั้งด้วย <code>app.setGlobalPrefix('api')</code>                                                                          | 2 (docs/learn/02-nestjs.md) |
| Pipe                      | ตรวจสอบและแปลงข้อมูลก่อนถึง controller เช่น <code>ValidationPipe</code> , <code>ParseIntPipe</code>                                         | 2 (docs/learn/02-nestjs.md) |
| DTO                       | class ที่นิยามหน้าตาของ body/query เช่น <code>CreateEmployeeDto</code>                                                                      | 2 (docs/learn/02-nestjs.md) |
| class-validator           | library ที่ให้เขียนกฎเป็น decorator เช่น <code>@IsString()</code> , <code>@Matches()</code>                                                 | 2 (docs/learn/02-nestjs.md) |
| <code>PartialType</code>  | สร้าง DTO ที่ทุกฟิลด์ไม่บังคับจาก DTO เดิม ใช้กับ <code>UpdateEmployeeDto</code>                                                            | 2 (docs/learn/02-nestjs.md) |
| Exception                 | สิ่งที่ <code>throw</code> เพื่อหยุดคำขอ เช่น <code>NotFoundException</code> แล้ว Nest แปลงเป็น <code>{ statusCode, message, error }</code> | 2 (docs/learn/02-nestjs.md) |
| Decorator metadata        | ข้อมูลชนิดของ constructor ที่ compiler ฝังไว้ให้ DI อ่าน esbuild/tsx ไม่สร้างให้                                                            | 2 (docs/learn/02-nestjs.md) |

ฐานข้อมูล

| คำ                                       | ความหมาย                                                                                                                    | un                                   |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| ORM                                      | เครื่องมือที่ให้เขียนโค้ดแทน SQL พร้อม type (โปรเจกต์นี้ใช้ Prisma)                                                         | 3 (docs/learn/03-database-prisma.md) |
| Schema<br>( <code>schema.prisma</code> ) | ไฟล์นิยามตารางและคอลัมน์                                                                                                    | 3 (docs/learn/03-database-prisma.md) |
| Prisma client                            | โค้ดที่ <code>prisma generate</code> สร้างจาก schema ใช้สั่ง <code>findMany</code> , <code>updateMany</code> ฯลฯ            | 3 (docs/learn/03-database-prisma.md) |
| Migration                                | ไฟล์ SQL ที่เปลี่ยนโครงสร้างฐานข้อมูลทีละขั้น เพิ่มได้อย่างเดียว                                                            | 3 (docs/learn/03-database-prisma.md) |
| Seed                                     | การใส่ข้อมูลตั้งต้น (5 records จาก Excel)                                                                                   | 3 (docs/learn/03-database-prisma.md) |
| Transaction                              | ชุดคำสั่งที่สำเร็จทั้งหมดหรือไม่เกิดอะไรเลย                                                                                 | 3 (docs/learn/03-database-prisma.md) |
| Index                                    | โครงสร้างช่วยค้นหาเร็ว เหมือนสารบัญท้ายหนังสือ                                                                              | 3 (docs/learn/03-database-prisma.md) |
| Trigram index                            | index ที่ช่วยค้นหาข้อความแบบ "มีคำนี้อยู่ข้างใน" ( <code>LIKE '%...%'</code> )                                              | 3 (docs/learn/03-database-prisma.md) |
| <code>ILIKE</code>                       | <code>LIKE</code> แบบไม่สนใจตัวพิมพ์เล็กใหญ่ Prisma <code>contains</code> + <code>mode: 'insensitive'</code> สร้างคำสั่งนี้ | 3 (docs/learn/03-database-prisma.md) |
| <code>numeric(12,2)</code>               | ชนิดตัวเลขทศนิยมแม่นยำ 12 หลัก ทศนิยม 2 ตำแหน่ง                                                                             | 6 (docs/learn/06-key-concepts.md)    |
| <code>Decimal</code>                     | object ที่ Prisma ใช้แทนค่า <code>numeric</code> แปลงเป็น string ด้วย <code>toFixed(2)</code>                               | 3 (docs/learn/03-database-prisma.md) |
| Date-only                                | วันที่ที่ไม่มีเวลาและ timezone เช่น <code>2023-01-15</code>                                                                 | 6 (docs/learn/06-key-concepts.md)    |
| SQL injection                            | การโจมตีด้วยการแทรก SQL ผ่านคำที่ผู้ใช้ส่งมา ป้องกันด้วย <code>parameter</code>                                             | 3 (docs/learn/03-database-prisma.md) |

OpenAPI

| คำ                  | ความหมาย                                                             | un                           |
|---------------------|----------------------------------------------------------------------|------------------------------|
| OpenAPI             | มาตรฐานเอกสาร API เป็น JSON/YAML บอก path, parameter, body, response | 4 (docs/learn/04-openapi.md) |
| Swagger             | ชื่อเดิมของ OpenAPI และชุดเครื่องมือ เช่น Swagger UI                 | 4 (docs/learn/04-openapi.md) |
| Swagger UI          | หน้าเว็บที่แสดงเอกสาร API และทดลองยิงได้ ( <code>/api/docs</code> )  | 4 (docs/learn/04-openapi.md) |
| Schema<br>(OpenAPI) | นิยามหน้าตาของข้อมูลชนิดหนึ่ง เช่น <code>CreateEmployeeDto</code>    | 4 (docs/learn/04-openapi.md) |

# Docker

| คำ          | ความหมาย                                                                                       | un                          |
|-------------|------------------------------------------------------------------------------------------------|-----------------------------|
| Image       | แม่พิมพ์ของโปรแกรมที่แพ็คเกจพร้อมรัน แกะข้างในไม่ได้ เช่น <code>postgres:17.11-bookworm</code> | 5 (docs/learn/05-docker.md) |
| Container   | image ที่กำลังรันอยู่                                                                          | 5 (docs/learn/05-docker.md) |
| Volume      | ที่เก็บข้อมูลถาวรนอก container                                                                 | 5 (docs/learn/05-docker.md) |
| Compose     | ไฟล์ YAML ที่บอกว่าจะเปิด container อะไร ตั้งค่าอย่างไร ( <code>compose.yaml</code> )          | 5 (docs/learn/05-docker.md) |
| Healthcheck | คำสั่งที่ Docker ใช้ตรวจว่า container พร้อมหรือยัง                                             | 5 (docs/learn/05-docker.md) |

# การทดสอบ

| คำ               | ความหมาย                                                                                  | un                           |
|------------------|-------------------------------------------------------------------------------------------|------------------------------|
| Unit test        | ทดสอบฟังก์ชันหรือกฎเล็ก ๆ ไม่ใช่ฐานข้อมูล                                                 | 7 (docs/learn/07-testing.md) |
| Integration test | ทดสอบ API ทั้งก่อนกับฐานข้อมูลจริง                                                        | 7 (docs/learn/07-testing.md) |
| E2E test         | ทดสอบผ่านเบราว์เซอร์จริงเหมือนผู้ใช้                                                      | 7 (docs/learn/07-testing.md) |
| Vitest           | ตัวรัน test ของ JavaScript/TypeScript                                                     | 7 (docs/learn/07-testing.md) |
| Supertest        | library ยิง HTTP เข้าแอป Nest ใน test                                                     | 7 (docs/learn/07-testing.md) |
| Playwright       | เครื่องมือควบคุมเบราว์เซอร์สำหรับ E2E                                                     | 7 (docs/learn/07-testing.md) |
| Fake timers      | <code>vi.useFakeTimers</code> ตั้งเวลาของ <code>Date</code> ใน test ให้ผลเหมือนเดิมทุกวัน | 7 (docs/learn/07-testing.md) |
| AC-xx            | รหัส acceptance criteria ใน PRD ที่ชื่อ test อ้างถึง                                      | 7 (docs/learn/07-testing.md) |

# เฉพาะโปรเจกต์นี้

| คำ                                                         | ความหมาย                                                                    | un                                   |
|------------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------|
| D-xx                                                       | รหัสการตัดสินใจใน decisions.md (docs/decisions.md)                          | 8 (docs/learn/08-docs-guide.md)      |
| PRD §x.y                                                   | อ้างถึงหัวข้อใน prd.md (docs/prd.md)                                        | 8 (docs/learn/08-docs-guide.md)      |
| <code>version</code>                                       | เลขรุ่นของแต่ละแถว เพิ่มทีละ 1 ทุกครั้งที่แก้ไขคู่กับ <code>If-Match</code> | 6 (docs/learn/06-key-concepts.md)    |
| Optimistic concurrency                                     | ไม่ล็อกข้อมูลตอนเปิดฟอร์ม แต่ตรวจ version ตอนบันทึกจนแล้วได้ 409            | 6 (docs/learn/06-key-concepts.md)    |
| "วันนี้" ตามกรุงเทพ                                        | วันที่ปัจจุบันในเขต <code>Asia/Bangkok</code> ใช้กับ Last Updated Date      | 6 (docs/learn/06-key-concepts.md)    |
| <code>DATABASE_URL</code> / <code>TEST_DATABASE_URL</code> | URL ของฐาน dev / ฐาน test ใน <code>.env</code>                              | 3 (docs/learn/03-database-prisma.md) |

กลับไป: สารบัญ (docs/learn/README.md)